

第 21 章: The Benchmarking Interlude (基准测试插曲)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 这是"函数与生成器"部分 (Part IV) 的收尾章。它不讲新语法, 而是把前面学过的函数、迭代工具 (for 循环、列表推导、map、生成器表达式与生成器函数) 放在一起做一次大型实战案例: 先自己编写计时工具, 再学习标准库的 timeit, 最后用这些工具比较 CPython 与 PyPy 的速度差异。同时, 本章复习函数部分最常见的陷阱 (gotchas), 是进入"模块"部分的过渡章。

21.1 开篇引言

英文原文

Now that we've fully explored function coding and iteration tools, we're going to take a short side trip to put both of them to work. This chapter closes out the function part of this book with a larger case study that at times the relative performance of the iteration tools we've met so far, in both standard Python and one of its alternatives. Along the way, this case study surv

eyes Python's code-timing tools, discusses benchmarking techniques in general, and develops code that's more realistic and useful than most of what we've seen up to this point. We'll also measure the speed of code we've used—data points that may or may not be significant, depending on your programs' goals. Finally, because this is the last chapter in this part of the book, we'll close with the usual sets of "gotchas" and exercises to help you start coding the ideas you've read about. First, though, let's have some fun with tangible Python code.

中文翻译

现在我们已经完整地探索了函数编码和迭代工具，接下来我们要短暂绕一下路，让两者都真正派上用场。本章以一个更大的案例研究来结束本书的函数部分：测量我们迄今遇到的各种迭代工具的相对性能——既在标准 Python 中，也在它的某个替代实现中。一路上，这个案例研究会全面考察 Python 的代码计时工具，讨论通用的基准测试（benchmarking）技术，并开发出比我们到目前为止见过的代码更贴近现实、更有用的代码。我们还会测量自己用过的代码的速度——这些数据点可能重要，也可能不重要，取决于你程序的目标。最后，因为这是本书这一部分的最后一章，我们照例会以一组“陷阱”（gotchas）和练习来收尾，帮助你开始动手编写你读到的这些想法。不过，先让我们用看得见摸得着的 Python 代码找点乐子。

深度理解

·核心概念：基准测试 (benchmarking) 是一门经验科学——"某某写法更快"这类结论必须在你自己的设备、你自己的 Python 版本、你自己的数据上验证才算数。本章把前面学的函数与迭代工具当作"被测对象"，把计时工具当作"测量仪器"。

·底层视角：Python 解释器内部不断被优化 (CPython 3.11 起引入了自适应特化解释器，未来还可能加入 JIT)，所以任何书上的速度数据都有保质期。作者特意提醒：把代码速度当"当前事实"而非"永恒真理"。

·设计原因：本书前面反复说过"列表推导和 map 有时比 for 快""生成器最慢"——这些说法必须被量化验证，否则只是传闻。这一章就是把这些说法变成可复现实验的过程。

·实际问题：选择工具时，除了"表达能力"这类主观因素，还有"性能"这类客观标准；工程师的职责之一就是基于证据做选择，而不是靠感觉。

·初学者误区：初学者最容易犯的错是用别人机器上的数据指导自己程序的优化。本章反复强调的限定条件——Python 版本、操作系统、测试代码类型——任何一项变了，结论都可能反转。

21.2 使用自制工具进行基准测试 (Benchmarking with Homegrown Tools)

英文原文

We've met quite a few iteration alternatives in this book. Like much in programming, they represent trade-offs—in terms of both subjective factors like expressiveness, and more objective criteria such as performance. Part of your job as a programmer and engineer is selecting tools based on factors like these. In terms of performance, this book has mentioned a few times that list comprehensions and map calls sometimes have a speed advantage over for loop statements. It has also noted that sorting speed varies with ordering, and the generator functions and expressions of the preceding chapter tend to be slower than all the others, though they minimize memory space requirements and don't delay the caller for result generation when there are many results to generate. All that is generally true today in common usage. That being said, benchmarking comes with some big caveats: both code structure and host architecture can influence speed arbitrarily; Python's performance can vary over time because its internals are constantly being changed and optimized; and the speed of alternative Pythons may differ widely. As noted in Chapter 2, for example, the standard CPython may adopt a standard JIT in the future which could change its speed in some contexts, and optimized Pythons like PyPy have very different performance profiles. In short, if you want to verify s

peed for yourself, you need to time your code, on your own device, with the Python or Pythons you plan to use, and in the here and now. That's a lot of qualifiers, but benchmarking is an empirical task.

中文翻译

我们在本书中已经遇到过相当多的迭代替代方案。和编程中的大多数事情一样，它们代表着权衡 (trade-offs) ——既涉及“表达力”这类主观因素，也涉及“性能”这类更客观的标准。作为程序员和工程师，你的部分工作就是根据这些因素挑选工具。就性能而言，本书已经提过几次：列表推导 (list comprehensions) 和 map 调用有时比 for 循环语句更快。书中也指出过：排序速度随数据顺序变化，上一章的生成器函数 (generator functions) 和生成器表达式 (generator expressions) 往往比所有其他方案都慢——尽管它们把内存空间需求降到最低，而且在结果很多时不会让调用方在生成结果这件事上等待。在今天的常见用法中，上述说法大体上都成立。话虽如此，基准测试附带着一些重大警告：代码结构和宿主架构都可能任意影响速度；Python 的性能会随时间变化，因为它的内部实现一直在被修改和优化；不同 Python 实现的速度可能差异巨大。例如，正如第 2 章所述，标准的 CPython 未来可能采用标准 JIT，这会在某些场景下改变它的速度；而像 PyPy 这样的优化版 Python 有着非常不同的性能画像 (performance profiles)。简言之，如果你想亲自验证速度，就需要在自己的设备上、用你计划使用的那个或那些 Python、在此时此刻给代码计时。限定条件很多，但基准测试本来就是一个经验性 (empirical) 任务。

深度理解

- 核心概念：迭代工具的取舍 = 主观维度（可读性、表达力）+ 客观维度（速度、内存）。本章只量化"速度"这一维，但选择工具时必须综合权衡——生成器虽然最慢，却是内存冠军。
- 底层视角：速度差异来自解释器内部实现的差别：map 的循环在 C 层运行，列表推导有专门的字节码优化，而生成器每次产出值都要保存/恢复执行状态（帧对象的状态机）。这些差异要到本章后面才会用数据揭示。
- 设计原因：作者反复加限定词（"一般来说" "今天的常见用法"），是因为 Python 内部变化太快，任何绝对论断都会过时。JIT 与否、版本差异、宿主架构都是变量。
- 实际问题：如果程序要处理海量数据，迭代方式的选择可能决定运行时间是几秒还是几分钟；但若数据量很小，任何方式都快，可读性才是首要的。
- 初学者误区：把"书上说 map 快"当成普适结论，不做任何验证。作者的忠告很直白：测速是一个经验任务——就像称体重，别人的秤不能代表你的体重。

21.3 计时模块：第一版 (Timer Module: Take 1)

英文原文

Luckily, Python makes it easy to time code with custo

m tools—though perhaps deceptively so. For example, to get the total time taken to run multiple calls to a function with arbitrary positional arguments, Example 21-1's function coded in a module file might suffice as a first cut.

中文翻译

幸运的是，用自定义工具给代码计时在 Python 中很容易——尽管也许容易得具有欺骗性。例如，要获得“用任意位置参数多次调用一个函数”所花的总时间，把 Example 21-1 的函数写进一个模块文件，作为第一版或许就够用了。

代码分析

```
"""
import time

def timer(func, *args):
    start = time.perf_counter()
    for i in range(100_000):
        func(*args)
    return time.perf_counter() - start
```

·解释器如何处理：time.perfcounter() 是标准库 time 模块提供的高分辨率时钟，专为测量短时耗时设计，跨平台可移植。它返回系统级时间（包含 sleep 的时间、受系统负载影响）。time.processtime() 则是“本进程 CPU 时间”，不含睡眠和系统其他活动，在某些场合更合适。本书上一版用的 time.clock() 已被弃用并从 Python 中删除。

·设计思想：这是“第一版”，作者故意让它犯下一系列经典错误，好引出改进版。它把重复次数硬编码为 100k；ran

ge(100000) 的构造与迭代开销被计入被测函数的时间；不支持关键字参数；不返回被测函数的结果，调用方无法验证函数是否真的正常工作；只给总时间，在繁忙的宿主机器上会波动。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

This function fetches time values from Python's standard-library time module, and subtracts the system start time from the stop time after running 100,000 calls to the passed-in function with the passed-in arguments. Time comes from time.perfcounter, which is defined to be a portable clock with the best resolution to measure a short duration. The difference between two calls is generally used for performance measurement, but includes time for sleeps and is system-wide ti

me. The alternative `time.process_time` is per-process CPU time, and may also be useful in some roles. See Python's manuals for more details; these calls replace the former and less portable `time.clock`, which was deprecated and dropped since this book's prior edition (breaking most examples here!). Apart from its time calls, the code in Example 21-1 is straightforward and uses tools we've already met. When used on this book's main development computer using macOS and CPython 3.12, its results are these in a REPL:

中文翻译

这个函数从 Python 标准库的 `time` 模块取时间值，在调用传入函数 10 万次之后，用停止时间减去系统起始时间。时间来自 `time.perfcounter`，它被定义为一个可移植的时钟，具有测量短时耗时的最佳分辨率。两次调用的差值通常用于性能测量，但它包含睡眠时间，并且是系统范围的时间。另一个选择 `time.process_time` 是每进程的 CPU 时间，在某些角色中也可能有用。更多细节见 Python 手册；这些调用取代了以前那个可移植性较差的 `time.clock`，后者在本书上一版之后已被弃用并移除（导致这里大多数示例崩掉！）。除了 `time` 调用，Example 21-1 的代码直截了当，用的都是我们已经见过的工具。在本书的主要开发电脑（macOS、CPython 3.12）上使用时，在 REPL 中得到如下结果：

英文原文 (REPL 运行结果)

```
>>> from timer0 import timer
>>> timer(pow, 2, 1000)                                # Time to call pow(2, 1...
0.08591239107772708
>>> timer(str.upper, 'hack' * 100)                     # Time to call 'hack....
0.04990812297910452
```

中文翻译 (REPL 运行结果)

```
>>> from timer0 import timer
>>> timer(pow, 2, 1000)                                # pow(2, 1000) 10 ...
0.08591239107772708
>>> timer(str.upper, 'hack' * 100)                     # 'hack...'.upper() ...
0.04990812297910452
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Though functional and simple, the preceding timer is also fairly limited, and deliberately exhibits some classic mistakes in both function design and benchmarki

ng. Among these, it: - Hardcodes the repetitions count at 100k - Charges the cost of range to the tested function's time - Doesn't support keyword arguments in the tested function - Doesn't give callers a way to verify that the tested function actually worked - Only gives total time, which might fluctuate on some busy host machines In other words, timing code is more complex than you might expect!

中文翻译

尽管功能可用、写法简单，上面的计时器也相当受限，而且作者故意让它表现出函数设计和基准测试方面的一些经典错误。其中包括：- 把重复次数硬编码为 10 万次- 把 range 的成本计进了被测函数的时间- 不支持被测函数使用关键字参数- 没有给调用方验证被测函数是否真正工作的途径- 只给出总时间，在忙碌的宿主机上可能会波动换句话说，计时代码比你想象的更复杂！

深度理解

·核心概念：好的计时工具必须做到"测量的纯粹性"——只测被测函数本身的开销，不掺入计时器自己（如 range 的构造、参数打包解包、额外的打印）的开销；还要"可验证性"——返回函数的结果，确认测的是正确的计算。

·底层实现：time.perfcounter 在 CPython 中最终落到操作系统提供的高分辨率时钟（Windows 上是 QueryPerformanceCounter，Linux 上是 clock_gettime(CLOCK_MONOTONIC)，macOS 上是 mach_absolute_time）。它

是"单调" (monotonic) 时钟——不会因为系统时间被调整而跳变，适合做差值测量。这就像用高精度秒表，而不是看墙上的挂钟。

·设计原因：第一版如此简陋，是教学上的"诱饵"：让你亲身体会"计时代码为何比想象复杂"。每一条缺陷都会会在第二版中被对应地修复，形成强烈的对比记忆。

·实际问题：真实世界里，重复次数必须可调（太快的一次调用测不出有效数据，太慢的又浪费时间）；被测函数可能既有位置参数又有关键字参数；忙碌机器上单次总时间会抖动，需要"取多轮最小值"来过滤。

·初学者误区：把 `time.clock()` 当标准写法（它已随 Python 3.8 移除）；只测一次就下结论；或者忘了验证被测函数真的干活了——如果函数因为 bug 瞬间返回，时间会"快"得让你误判。

21.4 计时模块：第二版 (Timer Module: Take 2)

英文原文

To be more general and accurate, let's rewrite the preceding section's code to define still simple but more useful timer utility functions we can both use to see how iteration alternative options stack up now, and apply to other timing needs in the future. These functions, in Example 21-2, are coded in a module file again so they can be used in a variety of programs, and h

ave docstrings giving some basic usage details that help and PyDoc can display; see Chapter 15 for tips on viewing the in-code documentation of this chapter's timing modules.

中文翻译

为了更通用、更准确，让我们重写上一节的代码，定义依然简单但更有用的 timer 工具函数——既用于评估各种迭代替代方案当下的表现，也可在未来应用到其他计时需求。Example 21-2 中的这些函数仍然写在一个模块文件里，这样就能被各种程序复用；它们带有 docstring，给出一些基本用法说明，help 和 PyDoc 可以显示这些文档；查看本章计时模块代码内文档的技巧见第 15 章。

代码分析

```
"""

N N "N "
    func
"""

import time
timer = time.perf_counter                # time.pr...

def once(func, *pargs, **kargs):         # func
    """
        func(...)
        (, )
    """
    start = timer()                      #
    result = func(*pargs, **kargs)       # func
    elapsed = timer() - start             #
    return (elapsed, result)              #
```

```

def total(reps, func, *pargs, **kargs):           # func
    """
    func(...)  reps
    (, )
    """
    total = 0                                     # range() ...
    for i in range(reps):
        time, result = once(func, *pargs, **kargs)
        total += time
    return (total, result)                        #

def bestof(reps, func, *pargs, **kargs):
    """
    func(...)  reps
    (, )
    """
    return min(once(func, *pargs, **kargs) for i in range(reps))

def bestoftotal(reps1, reps2, func, *pargs, **kargs):
    """
    reps1 [ func(...)  reps2 ]
    (, )
    """
    return min(total(reps2, func, *pargs, **kargs) for i in range(...

```

·解释器如何处理：pargs, kargs 是星号参数 (starred arguments)：在函数头部把调用时多余的位置参数收进元组、关键字参数收进字典；调用 func(pargs, kargs) 时再反向解包。这两步都有微小开销，但换来的是“任意参数形状的被测函数都能测”。once 返回 (elapsed, result) 元组：时间在前、结果在后——这个顺序是刻意的，因为后面 min 比较元组时“从左到右逐项比较”，时间项先比，就能按时间取最小。

·设计思想：把"测一次"做成原子操作（once），其他模式（total/bestof/bestoftotal）都建立在它之上。这样 range(reps) 的构造与迭代成本只发生在 total 的外层，不再混入被测函数的时间——这正是第一版"range 被计时"缺陷的修复。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Operationally, this module implements both total and best times, and a best of totals that combines the other two. In each mode, it times calls to any subject function that takes any positional and keyword arguments, by fetching the start time, calling the function with arbitrary arguments, and subtracting the start time from the stop time. Here are the salient points to notice about how this version addresses the shortcomi

ngs of its predecessor:- The repetitions count is no longer hardcoded, but passed in as a required argument (or arguments) before the test function and its own arguments, to allow repetitions to vary per call.- The range call's construction and iteration costs are no longer charged to timed functions, because timing has been factored out to the separate once function that times just the subject function.- Any number of both positional and keyword arguments for the timed function are now collected and unpacked with starred-argument syntax. They must be sent individually, not in a sequence or dictionary, though callers can unpack argument collections into individual arguments with stars in the top-level call.- All functions in this module return one of the timed function's return values so callers can verify that the function worked. The return value is provided in a two-item result tuple, along with the requested time result.- New best-of modes return minimum times to address fluctuations on the host device, per the next paragraph. This version's functions also support multiple use cases: once times a single call for simple cases; total runs many calls, to allow time to accumulate for short-lived functions; bestof returns the minimum time among all single calls, to filter out the impacts of other activity on the host; and bestoftotal selects the minimum of nested total-time tests, to both apply a best-of filter and run many calls for functions too fast to produce meaningful tim

es. Importantly, the min calls in the best-of variants work to select the best and lowest time, because Python compares collections recursively (from left to right) as we've learned, and result tuples begin with time: because it's first in these (time, result) tuples, time dominates and determines the min calls' results:

中文翻译

在操作层面，这个模块同时实现了 total（总计）和 best（最佳）时间，以及把两者结合的 best of totals（总计中的最佳）。每种模式下，它都是取开始时间、用任意参数调用被测函数、再用停止时间减去开始时间，从而对任何接受任意位置参数和关键字参数的被测函数计时。下面是要注意的要点，说明这一版如何解决前一版的缺陷：- 重复次数不再硬编码，而是作为必需参数传在测试函数及其自身参数之前，从而允许每次调用按需变化重复次数。- range 调用的构造与迭代成本不再计入被测函数，因为计时已被拆到独立的 once 函数，它只测被测函数本身。- 被测函数的任意数量的位置参数和关键字参数现在都用星号参数语法收集和解包。它们必须逐个发送，不能装在序列或字典里——不过调用方可以在顶层调用中用星号把参数集合展开成单个参数。- 这个模块的所有函数都返回被测函数的某个返回值，让调用方可以验证函数确实工作了。返回值连同请求的时间结果一起，放进一个二元组返回。- 新的 best-of 模式返回最小时间，用于应对宿主设备的波动，详见下一段。这一版的函数还支持多种使用场景：once 对单次调用计时，用于简单情形；total 运行多次调用，让时间在短命函数上得以累积；bestof 返回所有单次调用中的最小时间，过滤掉宿主机

上其他活动的影响；`bestoftotal` 在嵌套的总时间测试中取最小，既应用了 `best-of` 过滤，又对快得测不出有意义时间的函数运行多次调用。重要的是，`best-of` 变体中的 `min` 调用之所以能选出最佳（最低）时间，是因为正如我们学过的，Python 会递归地（从左到右）比较集合元素，而结果元组以 `time` 开头：因为时间在 `(time, result)` 元组中排第一，它占据主导，决定了 `min` 调用的结果：

英文原文 (REPL 演示)

```
>>> min(tup for tup in [(2.0, 3), (3.0, 3), (1.0, 3), (0.0, 3),...  
(0.0, 3)])
```

中文翻译 (REPL 演示)

```
>>> min(tup for tup in [(2.0, 3), (3.0, 3), (1.0, 3), (0.0, 3),...  
(0.0, 3)])
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

From a larger perspective, because these functions are coded in a module file, they become generally useful tools anywhere we wish to import them. Modules and imports were introduced in Chapter 3, and you'll learn more about them in the next part of this book; for now, simply import the module and call its function to use one of this file's timers. Its results on the same host and in a REPL are similar to its `timer0.py` predecessor, but are more robust:

中文翻译

从更大的视角看，因为这些函数写在模块文件里，它们就变成了我们想在哪儿用就在哪儿导入的通用工具。模块与导入在第3章引入，你将在本书下一部分了解更多；眼下，直接导入模块、调用函数就能用上这个文件里的计时器。在同一台主机、同一个 REPL 中，它的结果与前辈 `timer0.py` 类似，但更稳健：

英文原文 (REPL 运行结果)

```
>>> import timer                                # Import fi...
>>> help(timer)                                  # Display modu...
>>> reps, text = 100_000, 'hack' * 100
>>> timer.once(pow, 2, 1000)[0]                  # Not useful f...
6.182119250297546e-06
>>> timer.once(str.upper, text)                  # (time, resul...
(3.363005816936493e-06, 'HACKHACKHACK...etc...')
>>> timer.total(reps, pow, 2, 1000)[0]          # Compare to t...
0.08979405369609594
```

```
>>> timer.total(reps, str.upper, text)[0]           # (time, last ...
0.050884191412478685
>>> timer.bestof(50, pow, 2, 1000)[0]             # Not useful f...
1.6265548765659332e-06
>>> timer.bestof(50, str.upper, text)[0]           # (best time, ...
8.619390428066254e-07
>>> timer.bestoftotal(50, reps, pow, 2, 1000)[0]
0.07521858718246222
>>> timer.bestoftotal(50, reps, str.upper, text)[0]
0.03947464330121875
```

中文翻译 (REPL 运行结果)

```
>>> import timer                                     #
>>> help(timer)                                     #
>>> reps, text = 100_000, 'hack' * 100
>>> timer.once(pow, 2, 1000)[0]                     #
6.182119250297546e-06
>>> timer.once(str.upper, text)                     # (, )
(3.363005816936493e-06, 'HACKHACKHACK...etc...')
>>> timer.total(reps, pow, 2, 1000)[0]              # timer0 ...
0.08979405369609594
>>> timer.total(reps, str.upper, text)[0]           # (, ...
0.050884191412478685
>>> timer.bestof(50, pow, 2, 1000)[0]              #
1.6265548765659332e-06
>>> timer.bestof(50, str.upper, text)[0]           # (, ...
8.619390428066254e-07
>>> timer.bestoftotal(50, reps, pow, 2, 1000)[0]
0.07521858718246222
>>> timer.bestoftotal(50, reps, str.upper, text)[0]
0.03947464330121875
```

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

The last two calls here calculate the best-of-totals times—the lowest time among 50 runs, each of which computes the total time to call a function 100k times—roughly corresponding to the total times earlier in this listing, but repeated for a minimum that filters out host fluctuations (and sans an extra charge for range). The function used in these last two calls is really just a convenience that wraps total for better accuracy, but this is a common timing mode. Note that bestoftotal might replace its min of total with code like the following to nest total in bestof:

中文翻译

最后两次调用计算的是 best-of-totals 时间——50 轮运行中最低的一轮，其中每一轮都计算“调用函数 10 万次”的总时间——大致对应本清单前面的 total 时间，但重复取最小以过滤宿主波动（并且没有 range 的额外开销）。最后两

次调用用的这个函数实际上只是把 total 包一层以提升准确性的便利函数，但这是很常见的计时模式。注意，bestoftotal 也许可以用下面这样的代码把 total 嵌进 bestof，来代替它"对 total 取 min"的做法：

英文原文 (REPL 演示)

```
>>> timer.bestof(50, timer.total, reps, str.upper, text)
(0.07037258706986904, (0.039281503297388554, 'HACKHACKHACK...etc...'))
```

中文翻译 (REPL 演示)

```
>>> timer.bestof(50, timer.total, reps, str.upper, text)
(0.07037258706986904, (0.039281503297388554, 'HACKHACKHACK...etc...'))
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

But this isn't quite the same. For one thing, the result

is nested tuples, reflecting the nested calls. For another, this really times the entire total function, not just the subject function it runs. The net effect charges an extra admin-code overhead that's enough to skew the best-of time up, and make it larger than the best total time shown in the nested tuple. By using min of total instead, bestoftotal avoids timing this overhead skew. Subtle but true!

中文翻译

但这并不完全相同。其一，结果是嵌套元组，反映了嵌套的调用。其二，这实际上测的是整个 total 函数，而不只是它运行的那个被测函数。净效果是计入了额外的管理代码开销，足以把 best-of 时间抬高，让它比嵌套元组里显示的 best total 时间更大。改用"对 total 取 min"的写法后，bestoftotal 就避免了把这段开销计进时间。微妙，但千真万确！

深度理解

- 核心概念：一个成熟的计时器需要四种模式：once（单次）、total（多次累积，对付太快的函数）、bestof（多次取最小，过滤宿主抖动）、bestoftotal（嵌套组合：既取最小又累积次数）。每种模式应对一种真实测量场景。
- 底层实现：min 比较元组时按元素从左到右递归比较——(time, result) 中 time 在前，所以当 time 不同时 result 根本不会被比较。这是元组排序语义的巧妙利用：零额外代码就实现了"按时间取最小，结果作陪跑"。若把 result 放前面，min 就会按结果比较，逻辑立刻出错。

- 设计原因：所有函数都返回 (时间, 结果) 元组，是为了让调用方既能看时间又能验证正确性——计时和正确性必须绑定，否则测出的"快"可能是 bug 造成的。
- 实际问题：嵌套调用（把 timer.total 当被测函数传给 bestof）会把 total 自己的管理代码（for i in range(reps)、累加 total += time）也计进时间，导致数据偏高。这就是"测谁、就只该包含谁的开销"的测量原则。
- 初学者误区：以为"结果元组里有什么"无所谓；或者想当然认为嵌套组合等价。实际上参数顺序、元组元素顺序、嵌套层级都会改变测量语义——基准测试是极其挑剔的活。

21.5 计时运行器与脚本 (Timing Runner and Script)

英文原文

Now, to time iteration tool speed (our original goal), we'll write a script that defines and submits test functions to the timer module. To make it easy to code a variety of tests, let's first define a utility module that does the heavy lifting as a reusable intermediary. Example 21-3 defines a function named runner that takes any number of test functions as arguments and passes them off for timing to the bestoftotal function imported from Example 21-2.

中文翻译

现在，为了测试迭代工具的速度（我们最初的目标），我们要写一个脚本，定义测试函数并把它们交给 timer 模块。为了让各种测试写起来容易，先定义一个干重活的工具模块，作为可复用的中介。Example 21-3 定义了一个名为 runner 的函数，它接受任意数量的测试函数作为参数，并把它们交给从 Example 21-2 导入的 bestoftotal 函数去计时。

代码分析

```
""" timer.py """
import timer, sys

def runner(*tests):
    results = []
    print('Python', sys.version.split()[0], 'on', sys.platform) #...

    #
    for test in tests:
        besttime, result = timer.bestoftotal(10, 1000, test) # ...
        results.append(result)
        print(f'{test.__name__:<9}: ' # 9
              f'{besttime:.5f} => [{result[0]}...{result[-1]}]') # ...

    #
    print('Results differ!' #
          if any(result != results[0] for result in results[1:])
          else 'All results same.')
```

·解释器如何处理：sys.version.split()[0] 取出版版本号字符串（如 "3.12.2"），sys.platform 是平台名（如 'darwin' / 'win32'）。test.name 是函数的天然属性——函数对象上挂着一个名字属性。最后的输出用一个三元表达式做“全

等性验证": 把每个测试函数的结果与第一个对比, 任何一个不同就打印'Results differ!'. 这是对照实验的校验环节: 如果五种写法结果都不同, 说明被测代码本身有 bug, 计时毫无意义。

·设计思想: runner 是"测试框架"的雏形——把"计时"和"被测内容"分离。后面换被测函数、换测试语句, runner 一行不用改。

深度理解

·核心概念: 把握本节在整章知识链中的位置, 弄清它引入或巩固的关键术语与机制。

·底层实现: 对照 Python 运行时/迭代协议/对象模型, 理解代码实际如何被解释器处理。

·设计原因: Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题: 可在真实项目中用本节技术解决哪些任务; 何时该用、何时不该用。

·初学者误区: 最常见的混淆点 (与相似工具的边界、求值时机、可变/共享状态等)。

英文原文

Fine points here: this module's runner function displays context with sys tools documented in Python's manuals, and steps through all the passed-in functions, printing the name of each (as we've seen, this is a built-in attribute that gives a function's name). The test-

runner code also saves results to verify that they are all the same in a ternary expression at the very end of the process, to be sure we're comparing apples to apples. Last but not least, the script in Example 21-4 defines the actual tests to be timed and passes them to the imported runner of Example 21-3, which hands them off to the imported timer of Example 21-2. We'll run the file in Example 21-4 as a top-level script to time the relative speeds of the various iteration codings we've studied in this book so far.

中文翻译

这里的细节：这个模块的 runner 函数用 Python 手册中记载的 sys 工具显示运行环境，并遍历所有传入的函数，打印每个函数的 name（如我们所见，这是一个内建属性，给出函数的名字）。测试运行器代码还保存了各结果，在过程最末尾用一个三元表达式验证它们全部相同，确保我们在拿苹果和苹果比。最后但同样重要的是，Example 21-4 的脚本定义了真正要被计时的测试，并把它们传给 Example 21-3 导入的 runner，runner 再把它们交给 Example 21-2 导入的 timer。我们将把 Example 21-4 的文件作为顶层脚本运行，给本书迄今研究过的各种迭代写法计时，比较它们的相对速度。

代码分析

```

"""
from timer_runner import runner

repslist = list(range(10_000))          # range

def forLoop():                          # for + append
    res = []
    for x in repslist:
        res.append(abs(x))
    return res

def listComp():                         #
    return [abs(x) for x in repslist]

def mapCall():                         # map
    return list(map(abs, repslist))    # list()

def genExpr():                         #
    return list(abs(x) for x in repslist) # list()

def genFunc():                         #
    def gen():
        for x in repslist:
            yield abs(x)
    return list(gen())                 # list()

runner(forLoop, listComp, mapCall, genExpr, genFunc)

```

·解释器如何处理：三种惰性对象必须被"激活"——map 结果、生成器表达式、生成器函数返回的都是惰性迭代器，不消费就不会执行任何实际计算。用 list() 包装后，所有元素被逐一取出放入列表，实际工作才发生。repslist = list(range(10000)) 被提到模块顶层：range 本身是惰性序列，直接 for x in range(...) 会让"构造并遍历 range"的开销混进被测时间，先转成列表就能把这部分成本移出

测量窗口。作者实测：去掉这个 list 包装，测试时间会膨胀约 25%。

·设计思想：控制变量是基准测试的灵魂。五种写法除了“迭代机制”不同，其余一切（数据、强制求值方式、列表构建）都保持相同，这样测得的时间差异才能归因于迭代机制本身。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

This script tests five alternative ways to build lists of results. In combination with the test runner, its reported times reflect some 100 million steps for each of the 5 test functions—each builds a list of 10,000 items 1,000 times, and this process is repeated 10 times to get the best-of times. Applying this for each of the 5

test functions yields a whopping 500 million total steps for the script at large (impressive but reasonable on most machines these days). Notice how we have to run the results of the generator expression and function through the built-in list call to force them to yield all of their values; if we did not, we would just produce generators that never do any real work. We must do the same for the map result, since it is an iterable, on-demand object as well. For similar reasons, the inner loops' range result is hoisted out to the top of the module to remove its construction cost from total time, and wrapped in a list call so that its traversal cost isn't skewed by being a generator. This may be overshadowed by the cost of the inner iterations' loops, but it's best to remove as many variables as we can. For example, though range supports multiple scans, tests' times inflate by some 25% if its list wrapper is removed.

中文翻译

这个脚本测试了构建结果列表的五种替代方式。与测试运行器配合后，它报告的时间对应每个测试函数约 1 亿步的工作量——每个函数构建 1 万个元素的列表 1000 次，这个过程再重复 10 次以取得 best-of 时间。对 5 个测试函数逐一应用，整个脚本合计高达 5 亿步（在如今的多数机器上很惊人，但也算合理）。注意，我们必须把生成器表达式和生成器函数的结果送入内置 list 调用，强制它们产出所有值；如果不这么做，我们得到的只是从不干活的生成器。m

ap 结果也必须同样处理，因为它同样是一个按需迭代的可迭代对象。出于类似原因，内层循环的 range 结果被提升到模块顶部，以把它构造的成本移出总时间，并且用 list 调用包住，使遍历成本不会因为它是生成器而失真。这也许会被内层迭代循环的开销掩盖，但最好还是尽可能移除变量。例如，尽管 range 支持多次扫描，但如果去掉它的 list 包装，测试时间会膨胀约 25%。

深度理解

- 核心概念：控制变量法——被测的五种写法唯一区别是"迭代机制"。列表推导/生成器都用 list() 强制求值，range 提前转成列表，参数统一，返回结构统一（首尾元素一样）。

- 底层实现：惰性（lazy）对象只在你真正迭代它时才干活：map 返回的 map 对象、生成器表达式返回的生成器，都是一次性、按需产出的状态机。不 list() 包一层，计时器测到的是"创建一个不干活的生成器"的时间——约等于零，数据完全失真。

- 设计原因：range 提为模块级 repslist，是因为 range(10000) 每次执行都要构造对象，虽然便宜，但在 5 亿步的测量中任何一点无关开销都会污染结论。作者实测其影响高达 25%——测量时必须把"被测代码"以外的成本降到最低。

- 实际问题：生成器表达式必须在循环体外用 list() 收集，这正是它"延迟产出"特性的代价；这个代价本身也是它比列表推导慢的部分原因（后面会看到数据）。

·初学者误区：直接写 `map(abs, repylist)` 或裸生成器表达式去测时间，测出"0 秒"就以为 `map` 无敌——其实是生成器没被消费，什么也没发生。惰性对象必须被耗尽，测量才有意义。

21.6 迭代测试结果 (Iteration Results)

英文原文

When the top-level script of the prior section is run under the standard CPython 3.12 on the same macOS host, it prints the following with total times in seconds—after cueing the drum roll, that is:

中文翻译

上一节的顶层脚本在标准 CPython 3.12 下、于同一台 macOS 主机上运行时，打印出如下结果（以秒为单位的总时间）——先给个锣鼓预告，请欣赏：

英文原文 (运行结果)

```
$ python3 timer_tests.py
Python 3.12.2 on darwin
forLoop : 0.26035 => [0...9999]
listComp : 0.20781 => [0...9999]
mapCall  : 0.14399 => [0...9999]
genExpr  : 0.41133 => [0...9999]
genFunc  : 0.41203 => [0...9999]
All results same.
```


中文翻译 (运行结果)

```
$ python3 timer_tests.py
Python 3.12.2 on darwin
forLoop : 0.26035 => [0...9999]
listComp : 0.20781 => [0...9999]
mapCall  : 0.14399 => [0...9999]
genExpr  : 0.41133 => [0...9999]
genFunc  : 0.41203 => [0...9999]
All results same.
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

In short, map calls are faster than list comprehensions, which are quicker than for loops, and both generator expressions and functions come in last and roughly tied for slowest. Perhaps surprisingly, generator expressions run much slower than equivalent list compr

ensions today. As warned in the prior chapter, although wrapping a generator expression in a list call makes it functionally equivalent to a list comprehension, the internal implementations of the two expressions appear to differ: `return [abs(x) for x in repslist]` # 0.20 seconds `return list(abs(x) for x in repslist)` # 0.41 seconds: differs internally We're also effectively timing the list call for the generator test, but given that this does not seem to hamper `map`, it's likely moot. Though the exact cause for the difference would require deeper analysis (and probably source code spelunking), this seems to make sense given that the generator expression must do extra work to save and restore its state during value production; the list comprehension does not, and runs quicker here and in other tests ahead.

中文翻译

简言之，`map` 调用比列表推导快，列表推导比 `for` 循环快，生成器表达式和生成器函数并列垫底、大致并列最慢。也许令人意外的是，今天生成器表达式比等价的列表推导慢得多。正如上一章警告过的：虽然把生成器表达式用 `list` 调用包住，会让它在功能上与列表推导等价，但两种表达式在内部实现上似乎并不相同：`return [abs(x) for x in repslist]` # 0.20 秒 `return list(abs(x) for x in repslist)` # 0.41 秒：内部实现不同对生成器测试来说，我们实际上也把 `list` 调用的时间计了进去，但考虑到这似乎并没有拖累 `map`，很可能无关紧要。尽管造成差异的确切原因需要更深入的分析（

可能要钻源码)，但考虑到生成器表达式在产出值时必须做额外的保存/恢复状态的工作，这个差异就说得通了；列表推导不需要这些，因此在本文以及后面其他测试中跑得更快。

深度理解

·核心概念：在 CPython 3.12 上、这个测试场景里，速度排名是：map > 列表推导 > for 循环 > 生成器表达式 ≈ 生成器函数。"All results same." 这一行是结论可靠性的基石——五种写法产出相同的结果，速度差异才归因于实现方式。

·底层实现：map 的迭代循环整体运行在 C 层（每步只调用一次 Python 函数）；列表推导有专门的字节码（BUILDLIST、迭代器协议内联优化）；for 循环要频繁执行 append 方法查找和调用；生成器则每产出一次值都要保存/恢复帧状态（即“状态机”开销）。生成器表达式比列表推导慢一倍，正是这种每步状态保存/恢复成本的实证。

·设计原因：作者特意选择 abs（内置函数）作为每步操作，是想让五种写法处在“都必须调用函数”的同一起跑线上；在这个场景下 map 因 C 层循环获胜。

·实际问题：如果程序是 CPU 密集型的纯迭代，选 map 或列表推导能省三分之一以上的时间；但内存敏感场景（处理超大数据流）生成器的内存优势可以轻松抵消速度劣势。

·初学者误区：把这次测试的排名当成永恒真理——换 Python 版本、换操作符、换机器，排名都可能变（本章后面

PyPy、+ 运算两个测试就当场打脸）。

21.7 其他 Python 实现的结果 (Other Python's results)

英文原文

For comparison, following are the same tests' speed results on the same host using the current PyPy—the optimized Python implementation discussed in Chapter 2, whose current 7.3 release implements the Python 3.10 language. PyPy is roughly 5X quicker than CPython here (and up to 10X), though its timing results can vary widely if its JIT has not yet compiled code in full (and a future and currently hypothetical CPython JIT may or may not even the race):

中文翻译

作为对比，下面是在同一台主机上用当前 PyPy 运行同一组测试的速度结果——PyPy 是第 2 章讨论过的优化版 Python 实现，其当前的 7.3 版本实现的是 Python 3.10 语言。PyPy 在这里大约比 CPython 快 5 倍（最高可达 10 倍），不过如果它的 JIT 还没完全编译代码，计时结果可能波动很大（而且 CPython 未来那个目前还只是设想的 JIT 会不会把差距抹平，也未可知）：

英文原文（运行结果）

```
$ pypy3 timer_tests.py
Python 3.10.14 on darwin
forLoop : 0.04329 => [0...9999]
listComp : 0.01876 => [0...9999]
mapCall  : 0.04132 => [0...9999]
genExpr  : 0.08744 => [0...9999]
genFunc  : 0.08726 => [0...9999]
All results same.
```

中文翻译 (运行结果)

```
$ pypy3 timer_tests.py
Python 3.10.14 on darwin
forLoop : 0.04329 => [0...9999]
listComp : 0.01876 => [0...9999]
mapCall  : 0.04132 => [0...9999]
genExpr  : 0.08744 => [0...9999]
genFunc  : 0.08726 => [0...9999]
All results same.
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

On PyPy alone, list comprehensions win the title in this test today, but generators still lose soundly, and the fact that all of PyPy's results are so much quicker today seems the larger point here. On CPython, map is still quickest so far. Interestingly, these results are very different than they were in this book's prior edition on Windows under CPython 3.3—when generators placed in the middle of the pack between list comprehensions and for loops, for loops fared substantially worse than they do today, and the script had a different name for historical reasons:

中文翻译

仅就 PyPy 而言，今天的测试里列表推导夺魁，但生成器依然惨败；而今天 PyPy 的所有结果都快得多这件事，似乎是这里更大的要点。在 CPython 上，迄今 map 仍然最快。有趣的是，这些结果与本书上一版在 Windows 上、CPython 3.3 下的结果大不相同——那时生成器排在中间（介于列表推导和 for 循环之间），for 循环的表现比今天差得多，而且这个脚本出于历史原因还叫另一个名字：

英文原文（历史运行结果）

```
C:\code> c:\python33\python timeseqs.py
3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:57:17) [MSC v.1600 64 ...
forLoop : 1.33290 => [0...9999]
listComp : 0.69658 => [0...9999]
mapCall  : 0.56483 => [0...9999]
genExpr  : 1.08457 => [0...9999]
genFunc  : 1.07623 => [0...9999]
```

中文翻译（历史运行结果）

```
C:\code> c:\python33\python timeseqs.py
3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:57:17) [MSC v.1600 64 ...
forLoop : 1.33290 => [0...9999]
listComp : 0.69658 => [0...9999]
mapCall  : 0.56483 => [0...9999]
genExpr  : 1.08457 => [0...9999]
genFunc  : 1.07623 => [0...9999]
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

While these absolute times naturally reflect older and slower test hosts, these tools' relative performance has clearly changed in the last 12 years—and probably should be expected to do so again in another dozen!

中文翻译

这些绝对时间固然反映了更老、更慢的测试主机，但这些工具的相对表现在过去 12 年里显然已经改变——而且在下一个 12 年里，大概也应当预期它再次改变！

深度理解

·核心概念：同样的代码、不同的 Python 实现/版本，排名会变。PyPy 靠 JIT（即时编译）把热点循环编译成机器码，直接碾压 CPython 的解释循环；但在 PyPy 上获胜的是列表推导而不是 map——JIT 的特化优化改变了竞争格局。

·底层实现：PyPy 的 JIT 会“追踪”频繁执行的循环并编译为原生机器码，所以循环类代码提速巨大；但 JIT 尚未预热（warm-up）时结果会大幅波动——这就是为什么它的计时结果“变化大”。CPython 3.12 仍是解释执行（加部分特化），每步都要执行字节码循环。

·设计原因：作者摆出 CPython 3.3（2012 年）的历史数据，是为了证明一个方法论观点：“相对性能”本身是随时间漂移的。12 年前 for 循环最差、生成器居中；12 年后排名变了。没有永恒的排名，只有永恒的测量。

·实际问题：部署环境如果允许选解释器，PyPy 对 CPU 密集循环可能带来 5–10 倍提速；但 JIT 预热、兼容性、C 扩展（如 numpy 部分场景）都是现实约束，必须用自己的代码实测。

·初学者误区：拿 2012 年的博客文章或旧版书的数据优化 2026 年的代码；或者只测 CPython 就断言“生成器一定最慢”——在 PyPy 上生成器同样最慢，但 map 的排名就变了。结论必须标注三要素：Python 版本、机器、被测代码。

21.8 更多好戏：函数调用与 map (For more good times: Function calls and map)

英文原文

All of the foregoing is true as advertised, but watch what happens when Example 21-5 performs an inline operation on each iteration, such as a `+` expression, instead of calling a built-in function like `abs`. Only the test functions' operations (in bold) need to be modified here, and the imported and reused runner handles tests generically.

中文翻译

以上所有结论都如广告所言，但请看 Example 21-5 中每次迭代改为执行内联操作——比如 `+` 表达式，而不是调用 `abs` 这样的内置函数时会发生什么。这里只需要修改测试函

数的操作部分（原文中加粗的部分），导入复用的 runner 会泛化地处理测试。

代码分析

```
from timer_runner import runner

repslist = list(range(10_000))

def forLoop():
    res = []
    for x in repslist:
        res.append(x + 10)          #
    return res

def listComp():
    return [x + 10 for x in repslist] #

def mapCall():
    return list(map((lambda x: x + 10), repslist)) # lam...

def genExpr():
    return list(x + 10 for x in repslist)          #

def genFunc():
    def gen():
        for x in repslist:
            yield x + 10                #
    return list(gen())

runner(forLoop, listComp, mapCall, genExpr, genFunc)
```

·解释器如何处理：map 需要一个"可调用对象"作为第一参数，而加法不是一个可调用对象，所以只能塞一个 lambda x: x + 10。这意味着 map 的每次迭代都要调用一个 Python 层的 lambda 函数；而 for 循环、列表推导、生

成器里， $x + 10$ 是直接内联的字节码，没有函数调用。这一处差异彻底翻转了排名（见下）。

·设计思想：这个测试与上一个测试形成"对照实验组"：同一个 runner、同样的五次调用，只改每步操作的类型。于是数据差异可以干净地归因于"内置函数调用 vs Python 层函数调用 vs 内联操作"。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Now, map is slower than the for loop statements, despite the fact that the looping-statements version is larger in terms of code. This could either mean that the need to call a user-defined function makes map slower—or equivalently, that the lack of function calls makes the others quicker. On CPython 3.12 and the sam

e host as before:

中文翻译

现在，map 反而慢于 for 循环语句，尽管循环语句版本在代码量上更大。这要么意味着"需要调用用户定义函数"使 map 变慢——或者等价地说，是"没有函数调用"让其他写法变快。在 CPython 3.12、同一台主机上：

英文原文（运行结果）

```
$ python3 timer_tests2.py
Python 3.12.2 on darwin
forLoop : 0.28682 => [10...10009]
listComp : 0.24389 => [10...10009]
mapCall  : 0.49622 => [10...10009]
genExpr   : 0.44047 => [10...10009]
genFunc   : 0.44476 => [10...10009]
All results same.
```

中文翻译（运行结果）

```
$ python3 timer_tests2.py
Python 3.12.2 on darwin
forLoop : 0.28682 => [10...10009]
listComp : 0.24389 => [10...10009]
mapCall  : 0.49622 => [10...10009]
genExpr   : 0.44047 => [10...10009]
genFunc   : 0.44476 => [10...10009]
All results same.
```

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

These results have also been consistent in CPython: the prior edition's Python 3.3 results on a slower machine were again relatively similar, discounting test machine differences. Because the interpreter optimizes so much internally, performance analysis of Python code like this is a very tricky affair. Without numbers, it's virtually impossible to guess which method will perform the best; again, the best you can do is time your code with your test parameters. In this case, what we can say is that on this Python, using a user-defined lambda function in map calls seems to slow its performance disproportionately (though `+` is also slower than a trivial `abs` across the board), and that list comprehensions run quickest in this case (though slower than map in some others). List comprehensions seem consistently faster than for loops, but even this must be qualified—the list comprehension's relative speed

might be affected by its extra syntax (e.g., if filters), Python changes, and usage modes we did not time here.

中文翻译

这些结果在 CPython 上也保持了一致性：上一版书中在更慢机器上的 Python 3.3 结果，扣除测试机器差异后再次相对相似。由于解释器内部优化太多，对 Python 代码做这种性能分析是一件非常棘手的事。没有数据，几乎不可能猜出哪种方法会表现最好；再说一次，你最好的做法就是用你的测试参数给你的代码计时。就这个案例而言，我们能说的是：在这个 Python 上，在 map 调用里使用用户定义的 lambda 函数似乎不成比例地拖慢了它的性能（尽管 + 本身也比琐碎的 abs 慢，这是全面性的），而列表推导在这种情况下跑得最快（尽管在某些其他情况下比 map 慢）。列表推导似乎始终比 for 循环快，但即使这一点也必须加限定——列表推导的相对速度可能受它的额外语法（如 if 过滤）、Python 版本变化、以及我们这里设计时的使用模式影响。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

For deeper truth, Example 21-6 codes one last takeoff on our tests to apply a simple user-defined function in all five iterations timed. Again, we must only modify the relevant (and bold) bits of the test functions themselves.

中文翻译

为了更深的真相，Example 21-6 对我们的测试做了最后一次变奏：在所有五种被计时的迭代里都应用一个简单的用户自定义函数。同样，我们只需要修改测试函数本身相关（加粗）的部分。

代码分析

```
from timer_runner import runner

repslist = list(range(10_000))

def F(x): return x                                #

def forLoop():
    res = []
    for x in repslist:
        res.append(F(x))                          #
    return res
```

```

def listComp():
    return [F(x) for x in repslist] #

def mapCall():
    return list(map(F, repslist)) # F lambda

def genExpr():
    return list(F(x) for x in repslist) #

def genFunc():
    def gen():
        for x in repslist:
            yield F(x) #
    return list(gen())

runner(forLoop, listComp, mapCall, genExpr, genFunc)

```

·解释器如何处理：这次五种写法都调用同一个 Python 层函数 `F`，人人平等。`F` 是模块级名字，每次调用都要做全局名字查找再调用；`map(F, repslist)` 里 `F` 直接作为可调用对象传入，`map` 的 C 循环直接调用它。

·设计思想：这是“控制变量”的极致版本——把“操作”这个变量也统一成“调用一次用户函数”，从而隔离出“迭代机制”这一个变量的影响。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

When coded this way and run in CPython 3.12 on the same host again, map improves its relative times, and comes in second between list comprehensions and for loops—instead of being slower than all others as it was for +:

中文翻译

按这种方式编码并再次在 CPython 3.12、同一台主机上运行时，map 的相对时间改善了，排在列表推导和 for 循环之间的第二位——而不是像 + 测试那样比所有写法都慢：

英文原文（运行结果）

```
$ python3 timer_tests3.py
Python 3.12.2 on darwin
forLoop : 0.36206 => [0...9999]
listComp : 0.31181 => [0...9999]
mapCall  : 0.35479 => [0...9999]
genExpr  : 0.50531 => [0...9999]
genFunc  : 0.50290 => [0...9999]
All results same.
```

中文翻译 (运行结果)

```
$ python3 timer_tests3.py
Python 3.12.2 on darwin
forLoop : 0.36206 => [0...9999]
listComp : 0.31181 => [0...9999]
mapCall  : 0.35479 => [0...9999]
genExpr  : 0.50531 => [0...9999]
genFunc  : 0.50290 => [0...9999]
All results same.
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

That is, map may be slower simply because it requires function calls, and function calls are relatively slow in general. Since map can't avoid calling functions, it may lose by association, and the other iteration tools may win when they can use expressions instead. On t

he other hand, this hypothesis alone can't explain the better showing for map in the abs results of the first timertests.py: calling built-in functions may be a special and fast case for map in CPython (a theory supported by "Conclusion: Comparing tools" and ord at the end of this chapter's benchmarking safari). All this being said, performance should not be your primary concern when writing Python code—the first thing you should do to optimize Python code is to not optimize Python code! Write for readability and simplicity first, then optimize later, if and only if needed. It could very well be that any of the five iteration alternatives we've timed is quick enough for the data sets your program needs to process; if so, program clarity should be the chief goal.

中文翻译

也就是说，map 变慢可能仅仅因为它需要函数调用，而函数调用在总体上相对较慢。既然 map 无法避免调用函数，它可能因此“躺枪”落败，而其他迭代工具在能用表达式替代时就会获胜。另一方面，仅凭这个假设无法解释 map 在第一个 timertests.py 的 abs 测试中的更佳表现：在 CPython 中，调用内置函数对 map 来说可能是一个特殊且快速的案例（这一理论得到本章基准测试之旅末尾“结论：比较工具”和 ord 测试的支持）。话虽如此，写 Python 代码时性能不应该成为你的首要关注——优化 Python 代码的第一步是不要优化 Python 代码！先为可读性和简洁性而写，之后如果需要，再优化——而且只在确实需要时才优化。完

全可能，我们测过的五种迭代替代方案中任何一种对你的程序要处理的数据集来说都够快；如果真是这样，程序清晰度才应是首要目标。

深度理解

·核心概念：函数调用是昂贵操作。Python 的函数调用要做参数打包、栈帧创建/销毁、局部名字表建立等大量工作，比一个 + 字节码贵一个数量级。map 在"必须调用函数"的场合就失去了 C 层优势。

·底层实现：三个测试构成完整的证据链：(1) 每步调用内置 abs → map 最快（内置函数调用在 CPython 里被优化得极快，且 map 的循环在 C 层）；(2) 每步内联 + → map 必须调用 lambda，最慢；(3) 每步调用自定义 F → map 居中。同一份代码只改一个变量，结论一步步被钉死。

·设计原因：作者想教你一个方法论——单变量对照实验。三次测试就像三组对照实验，把"内置调用""内联运算""用户函数调用"三个因素逐一隔离。

·实际问题：这也是为什么"提前优化是万恶之源"：你无法凭空猜出哪个写法最快（三个测试里 map 分别排第 1、第 5、第 2 位！），必须先写清晰代码，再用自己的数据实测，只优化真正热点的代码。

·初学者误区：死记"map 最快"这种无条件的结论；或者反过来，测一次 + 场景就断言"map 很烂"。真实的工程教训是：性能结论必须有前提条件（操作类型、Python 实现、数据规模），脱离前提的优化建议都是玄学。

21.9 更多的模块改进 (More Module Mods)

英文原文

Our timer.py module works as designed, but it could be a bit more user-friendly. Most obviously, its functions require passing in repetition counts as first arguments, and provide no defaults for them—a minor point, perhaps, but less than ideal in a general-purpose tool. To do better, it could allow repetition counts to be passed in as keyword arguments with defaults, much the same way we did for the print emulators of Chapter 18. Here, though, these arguments' names would have to be distinct to avoid clashing with those of the timed function (e.g., reps instead of reps). While we're at it, the function object could also be a positional-only argument to prevent name func from clashing with a keyword argument of the same name in the timed subject. Example 21-7 codes the required mods (sans docs for space). Review Chapter 18's argument-ordering coverage for more insight.

中文翻译

我们的 timer.py 模块按设计工作正常，但它还可以更用户友好一些。最明显的是，它的函数要求把重复次数作为第一个参数传入，而且没有默认值——这也许是个小问题，但对一个通用工具来说不够理想。要做得更好，它可以让重复

次数以带默认值的关键字参数传入，就像我们在第 18 章给 print 模拟器所做的那样。不过在这里，这些参数的名字必须与众不同，以免与被测函数的参数名冲突（例如用 reps 而不是 rep）。既然要动手，函数对象还可以做成仅限位置（positional-only）参数，防止 func 这个名字与被测对象里同名的关键字参数冲突。Example 21-7 编写了所需的修改（为省空间省略了文档）。更多洞见请回顾第 18 章关于参数顺序的讲解。

代码分析

```
""" reps  func """
import time
timer = time.perf_counter

def once(func, /, *pargs, **kargs):
    start = timer()
    result = func(*pargs, **kargs)
    elapsed = timer() - start
    return (elapsed, result)

def total(func, /, *pargs, _reps=100_000, **kargs):
    total = 0
    for i in range(_reps):
        time, result = once(func, *pargs, **kargs)
        total += time
    return (total, result)

def bestof(func, /, *pargs, _reps=5, **kargs):
    return min(once(func, *pargs, **kargs) for i in range(_reps))

def bestoftotal(func, /, *pargs, _reps1=50, **kargs):
    return min(total(func, *pargs, **kargs) for i in range(_reps1))...
```

·解释器如何处理：/ 是 Python 3.8+ 的"仅限位置"参数分

隔符——/ 之前的参数（这里是 func）只能按位置传，绝不能写成 func=... 这样的关键字形式。这样被测函数即使有名为 func 的关键字参数，也不会与计时器自己的形参冲突。reps 放在 pargs 之后、kargs 之前，是"关键字专用"（keyword-only）参数——只能以 reps=100000 形式传入。注意 reps1（在 bestoftotal 中）是单独命名的：当 total 被调用时，调用方传入的 reps 无法匹配 total 的任何普通形参，于是被收集进 kargs 继续传给 total，total 再把它匹配给自己的 reps——作者特意用注释 # reps => 标注了这个传播链路。

·设计思想：这一版是对第 18 章"参数匹配规则"的一次活学活用：仅限位置 + 关键字专用 + 星号收集三个特性组合，让"计时器自己的参数"和"被测函数的参数"彻底隔离，永不冲突。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

This version is not backward compatible: it uses different names and modes for repetition arguments, which means the `timerrunner.py` we wrote earlier would require a minor edit to use it (see the examples package's `timer2.py`). Otherwise, it works the same—compare its output on the same host with `timer.py`'s results listed at Example 21-2:

中文翻译

这一版本不向后兼容：它对重复次数参数用了不同的名字和模式，这意味着我们之前写的 `timerrunner.py` 需要小改一下才能用它（见示例包里的 `timer2.py`）。除此之外，它的工作方式一样——在同一台主机上，把它的输出与 Example 21-2 列出的 `timer.py` 结果对比：

英文原文 (REPL 运行结果)

```
$ python3
>>> import timer2
>>> timer2.total(pow, 2, 1000, _reps=100_000)[0]
0.0865794476121664
>>> timer2.total(str.upper, 'hack' * 100, _reps=100_000)[0]
0.04620114527642727
>>> timer2.bestoftotal(pow, 2, 1000, _reps1=50, _reps=100_000)[0]
0.07179990829899907
>>> timer2.bestoftotal(str.upper, 'hack' * 100, _reps1=50, _reps=100_000)[0]
0.0393348871730268
```


中文翻译 (REPL 运行结果)

```
$ python3
>>> import timer2
>>> timer2.total(pow, 2, 1000, _reps=100_000)[0]
0.0865794476121664
>>> timer2.total(str.upper, 'hack' * 100, _reps=100_000)[0]
0.04620114527642727
>>> timer2.bestoftotal(pow, 2, 1000, _reps1=50, _reps=100_000)[0]
0.07179990829899907
>>> timer2.bestoftotal(str.upper, 'hack' * 100, _reps1=50, _reps=100_000)[0]
0.0393348871730268
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

This time, though, we can allow the functions' repetition defaults to apply or not:

中文翻译

不过这一次，我们可以让函数的重复次数默认值生效，也可以覆盖它：

英文原文 (REPL 运行结果)

```
>>> timer2.total(str.upper, 'hack' * 100)[0]
0.047992002684623
>>> timer2.bestoftotal(str.upper, 'hack' * 100)[0]
0.03935634717345238
>>> timer2.bestoftotal(str.upper, 'hack' * 100, _reps=10_000)[0]
0.00393257150426507
```

中文翻译 (REPL 运行结果)

```
>>> timer2.total(str.upper, 'hack' * 100)[0]
0.047992002684623
>>> timer2.bestoftotal(str.upper, 'hack' * 100)[0]
0.03935634717345238
>>> timer2.bestoftotal(str.upper, 'hack' * 100, _reps=10_000)[0]
0.00393257150426507
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Notice how the `reps` argument for `total`, if passed, in `bestoftotal` calls is propagated along in `\\kargs` because it's not matched otherwise. For more vetting of this, time user-defined functions with richer argument headers as in the following. Per the first four timings, passing keyword arguments to a timed function adds a small time cost for unpacking—an unavoidable overhead shared by the original `timer.py`, but irrelevant when collecting relative times:

中文翻译

注意 `bestoftotal` 调用里传给 `total` 的 `reps` 参数（如果传了的话）是如何经由 `\\kargs` 一路传播下去的——因为它无法被其他方式匹配。为了进一步检验这一点，给参数头更丰富的用户自定义函数计时，如下所示。根据前四次计时，向被测函数传关键字参数会带来一点点解包的额外时间成本——这是原版 `timer.py` 也同样承担的不可避免的开销，但在收集相对时间时无关紧要：

英文原文（REPL 运行结果）

```

>>> def f(a, b, c=88, d=99): return(a, b, c, d)
>>> f(1, 2)
(1, 2, 88, 99)
>>> timer2.bestoftotal(f, 1, 2, _reps1=50, _reps=100_000)
(0.014080120716243982, (1, 2, 88, 99))
>>> timer2.bestoftotal(f, 1, 2, c=66, d=77, _reps1=50, _reps=100_0...
(0.021575820166617632, (1, 2, 66, 77))
>>> timer2.bestoftotal(f, 1, 2, c=66, d=77, _reps1=50)
(0.021694606635719538, (1, 2, 66, 77))
>>> timer2.bestoftotal(f, 1, 2, c=66, d=77, _reps=100_000)
(0.021556788589805365, (1, 2, 66, 77))
>>> def f(a, *b, c=88, **d): return(a, b, c, d)
>>> f(1, 2, 3, c=66, d=77, e=88)
(1, (2, 3), 66, {'d': 77, 'e': 88})
>>> timer2.bestoftotal(f, 1, 2, 3, c=66, d=77, e=88)
(0.030859854072332382, (1, (2, 3), 66, {'d': 77, 'e': 88}))
>>> timer2.bestoftotal(f, 1, 2, 3, c=66, d=77, e=88, _reps1=10)
(0.030802161898463964, (1, (2, 3), 66, {'d': 77, 'e': 88}))
>>> timer2.bestoftotal(f, 1, 2, 3, c=66, d=77, e=88, _reps=10_000)
(0.0030745575204491615, (1, (2, 3), 66, {'d': 77, 'e': 88}))

```

中文翻译 (REPL 运行结果)

```

>>> def f(a, b, c=88, d=99): return(a, b, c, d)
>>> f(1, 2)
(1, 2, 88, 99)
>>> timer2.bestoftotal(f, 1, 2, _reps1=50, _reps=100_000)
(0.014080120716243982, (1, 2, 88, 99))
>>> timer2.bestoftotal(f, 1, 2, c=66, d=77, _reps1=50, _reps=100_0...
(0.021575820166617632, (1, 2, 66, 77))
>>> timer2.bestoftotal(f, 1, 2, c=66, d=77, _reps1=50)
(0.021694606635719538, (1, 2, 66, 77))
>>> timer2.bestoftotal(f, 1, 2, c=66, d=77, _reps=100_000)
(0.021556788589805365, (1, 2, 66, 77))
>>> def f(a, *b, c=88, **d): return(a, b, c, d)
>>> f(1, 2, 3, c=66, d=77, e=88)
(1, (2, 3), 66, {'d': 77, 'e': 88})

```

```
>>> timer2.bestoftotal(f, 1, 2, 3, c=66, d=77, e=88)
(0.030859854072332382, (1, (2, 3), 66, {'d': 77, 'e': 88}))
>>> timer2.bestoftotal(f, 1, 2, 3, c=66, d=77, e=88, _reps1=10)
(0.030802161898463964, (1, (2, 3), 66, {'d': 77, 'e': 88}))
>>> timer2.bestoftotal(f, 1, 2, 3, c=66, d=77, e=88, _reps=10_000)
(0.0030745575204491615, (1, (2, 3), 66, {'d': 77, 'e': 88}))
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Beyond this, custom benchmarking code is fun but open ended, and this section must stop here for space. As next steps, you might modify the timing script to measure the speed of set and dictionary comprehensions and their for equivalents (open questions we'll return to ahead); explore Python's profile and cProfile modules (tools that create full-program profiles instead of focused benchmarks); or explore Python's timei

t module, which works much like some of this chapter's homegrown code, and offers extra options—as you'll learn in the next section. NOTE — Decorator timers preview: Notice how we must pass functions into the timers manually here. In Chapter 39, we'll code decorator-based timer alternatives with which timed functions are called normally, but require extra @ preamble syntax where defined. Decorators may be more useful to instrument functions with timing logic when they are already being used within a larger system, and don't as easily support the specific test-call patterns assumed here—when decorated, every call to the function runs the timing logic, which is either a plus or minus depending on your goals.

中文翻译

除此之外，自定义基准测试代码有趣但开放性极强，本节出于篇幅必须到此为止。作为下一步，你可以修改计时脚本去测量集合和字典推导式及其 for 等价写的速度（这是开放问题，后面还会回到）；探索 Python 的 profile 和 cProfile 模块（它们生成的是全程序性能画像，而不是聚焦的基准测试）；或者探索 Python 的 timeit 模块——它的工作方式与本章自制代码很相似，但提供了额外选项——这正是你将在下一节学到的。注——装饰器计时器预告：注意这里我们必须手动把函数传进计时器。在第 39 章，我们将编写基于装饰器（decorator）的计时器替代方案：被测函数照常被调用，但在定义处需要额外的 @ 前缀语法。当函数已经在一个更大的系统中被使用时，装饰器可能更适合给

函数装上计时逻辑；但它们不容易支持这里假设的那种特定测试调用模式——一旦被装饰，对函数的每次调用都会运行计时逻辑，这在你目标不同时可能是优点也可能是缺点。

深度理解

·核心概念：参数模式的组合威力——/（仅限位置）、`args`（位置收集）、`reps=...`（关键字专用+默认值）、`kargs`（关键字收集）四个特性一起用，可以设计出“计时器参数与被测函数参数永不冲突”的 API。

·底层实现：`reps` 在 `bestoftotal` 调用中进不了 `total` 的显式形参（因为 `total` 里没有 `reps1` 这个位置），于是被兜进 `kargs` 字典，随调用一路传到 `total`，再由 `total` 的参数匹配机制识别为自己的 `reps`。这正是 `kargs` 收集器的“透传”（pass-through）用法——像是水管里的透明接头。

·设计原因：上一版把 `reps` 放在第一个位置参数，调用 `total(100000, pow, 2, 1000)` 需要记住参数顺序，不友好；改成带默认值的关键字参数后，不传就用默认值。工程工具追求“开箱即用”。

·实际问题：给被测函数传关键字参数比传位置参数慢约 50%（0.014 秒 vs 0.021 秒，见 REPL 数据）——这是 `kargs` 打包/解包的开销；但测量“相对速度”时这个固定开销对谁都一样，不影响结论。这正是“恒定开销不污染相对比较”的道理。

·初学者误区：以为参数名可以随便起。这里 `reps` 必须用下划线前缀，否则可能与被测函数的同名参数在 `kargs` 传

播中意外碰撞；func 必须做成仅限位置，否则被测函数若有关键字参数 func 就会互相遮蔽。API 设计中的"命名冲突"是最隐蔽的坑。

21.10 使用 Python 的 timeit 进行基准测试 (Benchmarking with Python's timeit)

英文原文

The preceding section used custom timing functions to compare code speed. As teased there, the Python standard library also ships with a module named `timeit` that can be used in similar ways, but offers added flexibility, with support for timing code strings in addition to functions, as well as a command-line mode. As usual in Python, it's important to understand fundamental principles like those illustrated in the prior section. Python's "batteries included" approach means you'll usually find precoded options for most goals, though you still need to know the ideas underlying them to choose and use them properly. Indeed, the `timeit` module is a prime example of this—it's had a history of being misused by newcomers who didn't yet understand the concepts it embodies. Now that we've learned the basics, though, let's move ahead to a tool that can automate much of our work.

中文翻译

上一节用自定义计时函数比较代码速度。正如那里预告的，Python 标准库还自带一个名为 `timeit` 的模块，可以用类似的方式工作，但提供了额外的灵活性：除了函数，它还能对代码字符串计时，还支持命令行模式。和 Python 中通常的情况一样，理解上一节演示的那些基本原理很重要。Python 的“自带电池”（batteries included）方针意味着大多数目标你都能找到预编好的方案，不过你仍然需要知道它们背后的理念，才能正确地选择和运用。事实上，`timeit` 模块就是最好的例子——它有着被不懂其内在概念的新手滥用的历史。不过，既然我们已经学了基础，现在就去看一个能替我们自动化大量工作的工具吧。

深度理解

- 核心概念：标准库工具 = 别人写好的实现 + 你需要理解的原理。没有上一节自建计时器的经验，你就无法判断 `timeit` 的 `min(repeat(...))` 为什么比裸测可靠，也容易误用。
- 底层实现：`timeit` 的核心机制与自建 timer 相同（`perfcounter` 计时、重复多轮取最小），但额外做了：代码字符串编译成函数、垃圾回收默认关闭、默认自动调节重复次数等（详见下节）。
- 设计原因：Python 把高频需求（测速）做成标准库，但作者强调“理解原理优先”——知其然更要知其所以然，才能避免工具用错场合。
- 实际问题：新人常犯的经典错误是拿 `timeit` 的默认参数

直接测毫秒级函数、或者把 `setup` 和被测语句混在一起，导致数据失真。本章先讲原理再上工具，正是为此。

·初学者误区：把 `timeit` 当黑盒"念咒语"。要知道它在内部做什么（构造 `def` 字符串、编译、循环执行），才能读懂它的输出（"best of 5"是什么意思、"usec per loop"怎么换算）。

21.11 timeit 基本用法 (Basic timeit Usage)

英文原文

Let's start with this module's fundamentals before leveraging them in larger scripts. With `timeit`, tests are specified by either callable objects or statement strings; the latter can hold multiple statements if they use ; separators or `\n` characters for line breaks, and spaces or tabs to indent statements in nested blocks (e.g., `\n\t`). Tests may also give setup actions, and can be launched from both command lines and API calls, and from both scripts and the REPL.

中文翻译

在更大的脚本里使用这个模块之前，先讲它的基础。用 `timeit` 时，测试既可以由可调用对象（callable objects）指定，也可以由语句字符串（statement strings）指定；后者可以用；分隔符或 `\n` 字符换行来容纳多条语句，用空格或

制表符给嵌套块中的语句缩进（例如 `\\n\\t`）。测试还可以给出 `setup`（初始化）动作，并且既可以从命令行启动，也可以从 API 调用启动，脚本和 REPL 里都行。

深度理解

·核心概念：timeit 接受两种被测对象：字符串（编译执行）和零参数的可调用对象。字符串更灵活（可含多条语句），但要受字符串转义规则约束；可调用对象没有转义问题，但被限制为无参数。

·底层实现：语句字符串最终会被包进一个合成的 `def` 函数体再编译执行，所以字符串里的 `\\n`、`\\t` 要按 Python 字符串字面量的规则写（比如 `\\n\\t` 表示“换行后缩进”）。

·设计原因：支持命令行和 API 两种模式，是为了覆盖“快速试一下”（命令行）和“写进脚本/CI”（API）两类使用场景。

·实际问题：一条语句如果含嵌套块（如 `while` 循环体），字符串里必须用 `\\n\\t` 表达缩进；漏掉一个转义字符，代码就编译失败。

·初学者误区：把代码字符串里的 `\\n` 当成真的回车——在 Python 字符串里它是转义序列，必须写成反斜杠加 `n`；不转义直接敲回车会产生语法错误。

21.12 API 调用模式 (API-calls mode)

英文原文

For example, the `timeit` module's `repeat` call returns a list giving the total time taken to run a test a number of times, for each of repeat runs—the min of this list yields the best time among the runs, and helps filter out system load fluctuations that can otherwise skew timing results artificially high (like our earlier `bestoftotal`). Code to be timed is passed to the `stmt` keyword (or first positional) argument, and may be a string or no-argument function. The following shows this call in action in REPLs on macOS, timing a list comprehension on both CPython 3.12 and the optimized PyPy implementation of Python described in Chapter 2 (as earlier, its tested 7.3 release implements the Python 3.10 language). The results here give the best total time in seconds among 5 runs that each execute the code string 1,000 times; the code string itself constructs a 1,000-item list of integers each time through (`timeit` also has reasonable defaults for repeat counts, but being explicit ensures comparability):

中文翻译

例如，`timeit` 模块的 `repeat` 调用返回一个列表：对每一轮 `repeat` 运行，给出“把测试运行 `number` 次”所花的总时间——对这个列表取 `min` 就能得到各轮中的最佳时间，并有助于过滤掉系统负载波动（否则它们会把计时结果人为抬高）（就像我们之前的 `bestoftotal`）。被测代码通过 `stmt`

关键字参数（或第一个位置参数）传入，可以是字符串，也可以是零参数函数。下面展示这个调用在 macOS 的 REPL 中的实际使用：在 CPython 3.12 和优化版 PyPy（第 2 章描述过的 Python 实现；如前所述，被测试的 7.3 版本实现的是 Python 3.10 语言）上分别给一个列表推导计时。这里的结果是 5 轮运行中最佳的总时间（秒），每轮都把代码字符串执行 1,000 次；代码字符串每执行一次就构建一个含 1,000 个整数的列表（timeit 对重复次数也有合理的默认值，但显式指定能确保可比性）：

英文原文（REPL 运行结果）

```
$ python3
>>> import timeit
>>> min(timeit.repeat(stmt='[x ** 2 for x in range(1000)]', number...
0.05187674192711711
$ pypy3
>>>> import timeit
>>>> min(timeit.repeat(stmt='[x ** 2 for x in range(1000)]', numbe...
0.00252467580139637
```

中文翻译（REPL 运行结果）

```
$ python3
>>> import timeit
>>> min(timeit.repeat(stmt='[x ** 2 for x in range(1000)]', number...
0.05187674192711711
$ pypy3
>>>> import timeit
>>>> min(timeit.repeat(stmt='[x ** 2 for x in range(1000)]', numbe...
0.00252467580139637
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

You'll notice that PyPy checks in at 20X faster than CPython 3.12. This is a small artificial benchmark, of course, but stunning nonetheless, and reflects a relative speed ranking that is generally supported by other tests run in this book (though CPython will still beat PyPy on some types of code ahead, and again, may improve with a future JIT). To be fair, this particular test measures the speed of both a list comprehension and integer math, but integer math is ubiquitous in Python code, and PyPy's win is larger on noninteger tests (try squaring a float to see for yourself). These results also differ from the preceding section's relative version speeds, where PyPy was some 10X quicker for lis

t comprehensions. Apart from the different type of code being timed here, the different coding structure inside timeit may have an effect too—for code strings like those tested here, timeit builds, compiles, and executes a function def statement string that embeds the test string, thereby avoiding a function call per inner loop. This is irrelevant from a relative-speed perspective, though: times from the same given tool are comparable. You'll also notice that code to be timed is a string here. As you'll see ahead, this requires manually coded line breaks and indentation in some usage modes, and all code to be timed this way must conform to Python's rules for string literals, quotes, and escapes. For instance, a code string cannot embed the same quotes used to enclose it without also escaping them. You can avoid this potential downside by timing callables, though they're limited to zero arguments.

中文翻译

你会注意到 PyPy 以比 CPython 3.12 快 20 倍的成绩登场。这当然是个小型的人工基准，但依然令人震撼，而且反映了一个大体被本书其他测试支持的速度排名（不过在后面某些类型的代码上 CPython 仍然会赢过 PyPy，而且如前所述，CPython 可能会靠未来的 JIT 进步）。公平地说，这个特定测试同时测了列表推导和整数运算的速度，但整数运算在 Python 代码中无处不在，而且 PyPy 在非整数测试上赢面更大（你可以自己试试对一个浮点数求平方）。这些结果与上一节“PyPy 对列表推导约快 10 倍”的相对速度也不

同。除了被测代码类型不同之外，timeit 内部不同的代码构造方式也可能有影响——对这里这类代码字符串，timeit 会构建、编译并执行一个嵌入了测试字符串的函数 def 语句字符串，从而避免了内层循环每轮一次的函数调用。不过，从相对速度的角度看这无关紧要：同一工具测得的时间是可比的。你还会注意到，这里的被测代码是字符串。正如你接下来会看到的，这在某些使用模式下需要手工编写换行和缩进，而且所有这样被测的代码都必须符合 Python 字符串字面量、引号和转义的规则。例如，代码字符串不能嵌入与包裹它的引号相同的引号，除非加以转义。你可以通过测可调用对象来避开这个潜在缺点，不过它们被限制为零参数。

深度理解

- 核心概念：repeat(stmt, number, repeat) 的三层语义——stmt 是被测语句，number 是每轮执行的次数，repeat 是轮数；对返回列表取 min，得到"最佳轮"。这与你自建的 bestoftotal(50, 100000, ...) 完全同构。
- 底层实现：timeit 把 stmt 字符串嵌入一个合成的 def 函数体内编译执行，这样内层循环不会再有每次迭代的函数调用开销（比直接 exec 字符串或逐条执行更贴近真实场景）。这就是"为什么自建 timer 与 timeit 结果略有不同"的机理之一。
- 设计原因：字符串模式的存在，是为了能在命令行里直接写代码测速；它的代价是转义规则的折磨。两害相权，timeit 两种模式都给。
- 实际问题：PyPy 对这份代码快 20 倍，但上一节同是 PyPy 只有 10 倍——比例本身在不同测试里都不同，更别说

绝对时间。这再次证明：倍数也不是常数。

·初学者误区：把 `stmt='...'` 里的单引号与被测代码的引号混用导致语法错误（例如 `stmt='"hack' 5"` 外层双引号包内层单引号才行）；或者想当然认为 `timeit` 与自制 `timer` 的绝对数值应该一致——不同工具、不同结构，绝对数不同很正常，比相对排名即可。

21.13 命令行模式 (Command-line mode)

英文原文

The `timeit` module also can be run as a script from a command line—either by explicit pathname, or, more portably, automatically located on the module search path with Python's `-m` switch (we used this earlier to launch `PyDoc` in Chapter 15 and `IDLE` in Chapter 3). In this mode, you'll need to quote or escape Python code in the command line per your console shell's rules; double quotes are generally portable, but this also qualifies as a potential downside. Also in this mode, `timeit` reports the average time for a single `-n` loop, in either "usec" microseconds (millionths), "msec" milliseconds (thousandths), "sec" seconds, or "nsec" nanoseconds; to compare results here to the total time values reported by API calls, multiply by the number of loops run—51 usec here 1,000 loops is 51k usec, 51 msec, and 0.051 seconds in total time. For CPython 3.12 on macOS:

中文翻译

timeit 模块也可以作为脚本从命令行运行——要么用显式路径名，要么更可移植地通过 Python 的 `-m` 开关在模块搜索路径上自动定位它（我们之前用这个开关在第 15 章启动过 PyDoc、在第 3 章启动过 IDLE）。在这种模式下，你需要按照控制台 shell 的规则对命令行中的 Python 代码加引号或转义；双引号通常是可移植的，但这也算一个潜在的缺点。同样在这种模式下，timeit 报告的是单次 `-n` 循环的平均时间，单位是 "usec" 微秒（百万分之一秒）、"msec" 毫秒（千分之一秒）、"sec" 秒或 "nsec" 纳秒；要把它与 API 调用报告的总时间对比，需要乘以运行的循环次数——这里的 51 usec 1,000 次循环 = 51k usec = 51 msec = 总时间 0.051 秒。macOS 上 CPython 3.12 的结果：

英文原文（运行结果）

```
$ python3 -m timeit -n 1000 "[x ** 2 for x in range(1000)]"
1000 loops, best of 5: 51.9 usec per loop
$ python3 -m timeit -n 1000 -r 50 "[x ** 2 for x in range(1000)]"
1000 loops, best of 50: 51.8 usec per loop
```

中文翻译（运行结果）

```
$ python3 -m timeit -n 1000 "[x ** 2 for x in range(1000)]"
1000 loops, best of 5: 51.9 usec per loop
$ python3 -m timeit -n 1000 -r 50 "[x ** 2 for x in range(1000)]"
1000 loops, best of 50: 51.8 usec per loop
```

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

As another example, we can use command lines to verify that choice of timer call doesn't impact speed comparisons run in this chapter so far. `timeit` uses `time.perfcounter` by default but its `-p` flag instructs it to instead use `time.process_time`, both discussed earlier (spoiler: as tested, there's no discernible difference):

中文翻译

再举一个例子，我们可以用命令行来验证“计时函数的选择”不会影响本章到目前为止的速度比较。`timeit` 默认使用 `time.perfcounter`，但它的 `-p` 标志指示它改用 `time.process_time`，两者前面都讨论过（剧透：实测没有可辨别的差异）：

英文原文（运行结果）

```
$ python3 -m timeit -p -n 1000 -r 50 "[x ** 2 for x in range(10...  
1000 loops, best of 50: 51.8 usec per loop
```

中文翻译 (运行结果)

```
$ python3 -m timeit -p -n 1000 -r 50 "[x ** 2 for x in range(10...  
1000 loops, best of 50: 51.8 usec per loop
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

We can also use timeit to see how PyPy stacks up again—but we're in for a bit of a surprise:

中文翻译

我们也可以用 timeit 再看看 PyPy 的表现——但我们即将迎来一点意外：

英文原文 (运行结果)

```
$ pypy3 -m timeit -n 1000 -r 50 "[x ** 2 for x in range(1000)]"
WARNING: timeit is a very unreliable tool. use pyperf or something
else for real measurements
pypy3 -m pip install pyperf
pypy3 -m pyperf timeit -n '1000' -r '50' '[x ** 2 for x in range(1...
-----
1000 loops, average of 50: 1.54 +- 0.568 usec per loop (using stan...
```

中文翻译 (运行结果)

```
$ pypy3 -m timeit -n 1000 -r 50 "[x ** 2 for x in range(1000)]"
WARNING: timeit is a very unreliable tool. use pyperf or something
else for real measurements
pypy3 -m pip install pyperf
pypy3 -m pyperf timeit -n '1000' -r '50' '[x ** 2 for x in range(1...
-----
1000 loops, average of 50: 1.54 +- 0.568 usec per loop (using stan...
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

As you can see, PyPy has customized command-line mode in its version of timeit to both emit a subjective redirect to an alternative timer, as well as modify the result display to show averages instead of minimums. These are both opinionated mods made since this book's prior edition, though perhaps understandable for a tool so focused on (and sensitive to) benchmark results. See the web for more on the suggested pyperf; it's not part of Python's standard library and requires a separate install, but may offer more advanced timing options that may or may not apply to your goals.

中文翻译

如你所见，PyPy 在其版本的 timeit 中定制了命令行模式：既输出一段主观的指引（建议改用替代计时器），又把结果显示改成了平均值而不是最小值。这是本书上一版之后做出的两项“带立场”的修改，不过对于一个如此聚焦于（且敏感于）基准测试结果的工具来说，也许可以理解。关于被推荐的 pyperf，请在网上了解更多；它不是 Python 标准库的一部分，需要单独安装，但可能提供更高级的计时选项，这些选项是否适合你的目标则要另说。

深度理解

·核心概念：-m timeit 借助 Python 的 -m 模块运行机制，把 timeit 当脚本执行；-n 是每轮循环次数，-r 是轮数

- 报告格式"best of 5"告诉你它取的是最小值（-r 轮中最小的一轮），单位是"每循环一次"的平均耗时。
- 底层实现：命令行与 API 的数值换算关系： $51.9 \text{ usec} \times 1000 \text{ loops} \approx 0.0519 \text{ 秒}$ 。-p 切换计时函数，实测两种时钟（perfcounter 与 processtime）几乎无差别，证明本 CPU 密集测试中两者等价。
- 设计原因：PyPy 团队认为 min 掩盖了 JIT 的不稳定性，改为显示"平均值 $\pm$ 标准差"（ $1.54 \pm 0.568 \text{ usec}$ ）并推荐 pyperf——这是不同实现方对"如何报告数据更诚实"的不同哲学。你的结论取决于你信哪种统计口径。
- 实际问题：跨 Python 实现做命令行基准测试时，输出格式可能完全不同（PyPy 的警告和平均值模式），自动化解析输出时要格外小心——这正是后面 pybench 的 -a 模式"几乎不可用"的根源。
- 初学者误区：把 "51.9 usec per loop" 直接当作 API 模式的总时间比较——单位完全不同（每循环 vs 总时间）；或者不知道 -r 默认是 5，-n 不指定时 timeit 会自动推算循环次数。

21.14 处理多行语句 (Handling multiline statements)

英文原文

Happily, timeit's API mode is still opinion-free today. To time larger blocks of code in API-call mode, either

load code from a file, use a triple-quoted block string, or use newlines and tabs or spaces to satisfy Python's syntax. Because you pass Python string objects to a Python function in this mode, there are no shell considerations, though be careful to handle nested quotes with escapes or mixed quotes if needed. The following, for instance, times Chapter 13 loop alternatives in CPython 3.12; you can use the same pattern to time the file-line-reader alternatives in Chapter 14:

中文翻译

令人欣慰的是，timeit 的 API 模式至今仍然“不带立场”。要在 API 调用模式下给更大的代码块计时，可以从文件加载代码、用三引号块字符串，或者用换行加制表符/空格来满足 Python 的语法。因为这种模式下你把 Python 字符串对象传给 Python 函数，没有 shell 的顾虑；不过需要注意用转义或混合引号处理嵌套引号。例如，下面在 CPython 3.12 上给第 13 章的循环替代方案计时；你可以用同样的模式给第 14 章的文件行读取替代方案计时：

英文原文 (REPL 运行结果)

```
$ python3
>>> import timeit
>>> min(timeit.repeat(number=100_000, repeat=5,
    stmt='L = [1, 2, 3, 4, 5]\nfor i in range(len(L)): L[i] +=...',
0.028153221122920513
>>> min(timeit.repeat(number=100_000, repeat=5,
    stmt='L = [1, 2, 3, 4, 5]\nfor i in range(len(L)): L[i] +=...',
0.03337613819167018
>>> min(timeit.repeat(number=100_000, repeat=5,
```



```
stmt='L = [1, 2, 3, 4, 5]\nM = [x + 1 for x in L]'))  
0.021507765166461468
```

中文翻译 (REPL 运行结果)

```
$ python3  
>>> import timeit  
>>> min(timeit.repeat(number=100_000, repeat=5,  
    stmt='L = [1, 2, 3, 4, 5]\nfor i in range(len(L)): L[i] +=...  
0.028153221122920513  
>>> min(timeit.repeat(number=100_000, repeat=5,  
    stmt='L = [1, 2, 3, 4, 5]\ni=0\nwhile i < len(L):\n\tL[i] ...  
0.033337613819167018  
>>> min(timeit.repeat(number=100_000, repeat=5,  
    stmt='L = [1, 2, 3, 4, 5]\nM = [x + 1 for x in L]'))  
0.021507765166461468
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

To run multiline statements like these in command-line mode, appease your shell by passing each statement line as a separate argument, with whitespace for indentation—`timeit` concatenates all the lines together with a newline character between them, and later re-indents for its own statement-nesting purposes. Leading spaces may work better for indentation than tabs in this mode due to shell variability, and be sure to quote the code arguments if required by your shell (the first of the following was line-split here to fit, but must be input on a single line in some shells):

中文翻译

要在命令行模式下运行这样的多行语句，请安抚你的 shell：把每条语句行作为单独的参数传入，用空白做缩进——`timeit` 会把所有行用换行符连接起来，然后为了它自己的语句嵌套目的重新缩进。由于 shell 的差异，在这种模式下用前导空格做缩进可能比制表符更可靠；如果 shell 需要，务必给代码参数加引号（下面第一个命令在这里为了排版被分成了多行，但在某些 shell 里必须在单行内输入）：

英文原文（运行结果）

```
$ python3 -m timeit -n 100000 -r 5 "L = [1,2,3,4,5]" "i=0" "whi...  
  " L[i] += 1" " i += 1"  
100000 loops, best of 5: 332 nsec per loop  
$ python3 -m timeit -n 100000 -r 5 "L = [1,2,3,4,5]" "M = [x + 1 f...  
100000 loops, best of 5: 218 nsec per loop
```

中文翻译 (运行结果)

```
$ python3 -m timeit -n 100000 -r 5 "L = [1,2,3,4,5]" "i=0" "whi...  
  " L[i] += 1" " i += 1"  
100000 loops, best of 5: 332 nsec per loop  
$ python3 -m timeit -n 100000 -r 5 "L = [1,2,3,4,5]" "M = [x + 1 f...  
100000 loops, best of 5: 218 nsec per loop
```

深度理解

·核心概念：多行代码在两种模式下有不同的表达方式：API 模式用 `\n` 转义序列（字符串对象直接进 Python），命令行模式用“每个参数一行、timeit 自动拼接”。注意命令行模式里 `" L[i] += 1"` 行首的空格就是缩进。

·底层实现：命令行的多个参数被 timeit 用 `\n` 连接成一个多行字符串，然后它会对合成的 `def` 代码重新缩进（每条语句嵌套深度由它自己管理）。332 nsec（纳秒）意味着这个循环每次只有 0.332 微秒——小到刷新你对“Python 慢”的认知。

·设计原因：为什么要这么麻烦地支持多行？因为真实代码（while 循环、多个语句）几乎都不是单行的；不支持多行，timeit 就只能测玩具代码。

·实际问题：字符串里写 `\t` 缩进与命令行里用空格缩进，都是为了让嵌套块合法；API 模式里漏掉 `\n\t` 就会 `IndentationError`。这是新手使用 timeit 最常碰壁的地方。

·初学者误区：把 `\n` 当普通字符写进 stmt（写成 `L = [...]` 或 `' + 'for ...'` 之类），或者忘记 while 循环体需要额外一层 `\t` 缩进。字符串转义是这门课的基本功，在这里直接决定

能否测成功。

21.15 timeit 的其他用法模式 (Other timeit usage modes)

英文原文

The timeit module also allows you to provide setup code that is run in the main statement's scope, but whose time is not charged to the main statement's total—useful for initialization code you wish to exclude from total time, such as imports of required modules and builds of test data. Because they're run in the same scope, any names created by setup code are available to the main test statement; names defined in the interactive shell generally are not. To specify setup code, use a `-s` in command-line mode (or many of these for multiline setups) and a setup argument string in API-call mode. This can focus tests more sharply, as in the following, whose second command splits list initialization off to a setup statement to time just iteration in command-line mode. As a general rule of thumb, though, the more code you include in a test statement, the more applicable its results will generally be to realistic code:

中文翻译

timeit 模块还允许你提供 setup 代码：它为主语句的作用域中运行，但它的时间不计入主语句的总时间——这对你想从总时间里排除的初始化代码很有用，比如导入所需模块、构建测试数据。由于它们在同一作用域中运行，setup 代码创建的任何名字对主测试语句都是可见的；而交互式 shell 里定义的名字一般不可见。指定 setup 代码的方式：命令行模式用 -s（多行 setup 可以多次使用），API 调用模式用 setup 参数字符串。这可以让测试更聚焦，如下所示：第二条命令把列表初始化拆到 setup 语句里，从而在命令行模式下只测迭代部分。不过一般来说，一个经验法则：测试语句里包含的代码越多，其结果通常越接近真实代码：

英文原文（运行结果）

```
$ python3 -m timeit -n 100000 -r 5 "L = [1,2,3,4,5]" "M = [x + ...  
100000 loops, best of 5: 211 nsec per loop  
$ python3 -m timeit -n 100000 -r 5 -s "L = [1,2,3,4,5]" "M = [x + ...  
100000 loops, best of 5: 175 nsec per loop
```

中文翻译（运行结果）

```
$ python3 -m timeit -n 100000 -r 5 "L = [1,2,3,4,5]" "M = [x + ...  
100000 loops, best of 5: 211 nsec per loop  
$ python3 -m timeit -n 100000 -r 5 -s "L = [1,2,3,4,5]" "M = [x + ...  
100000 loops, best of 5: 175 nsec per loop
```

深度理解

·核心概念：setup 把“初始化”与“被测主体”分开计时：-s "L = [...]" 之后主语句只测 M = [x + 1 for x in L]。同样的代码，一次拆一次不拆，结果是 175 vs 211 nsec——

差出的 36 nsec 正是列表构造的时间。

- 底层实现：setup 与主语句被编译进同一个合成函数体（setup 在前、stmt 在后），所以 setup 创建的 L 对主语句可见；而 REPL 里敲的全局名则不在该函数的作用域链上，所以不可见。这正是“为什么 -s 里必须导入模块”的原因。

- 设计原因：timeit 提供了 setup 能力，但作者同时强调反直觉的经验法则——测的代码越“真实”（包含初始化），结果越有参考价值。极端“纯净”的微基准往往与真实程序相去甚远。

- 实际问题：如果被测函数需要 import，必须在 setup 里 import（-s "from mymod import func"），否则每次运行都会找不到名字；而把 import 放主语句里测，则会连 import 的耗时一起算进去。

- 初学者误区：想当然地以为 REPL 里定义好的名字能直接用于 stmt 字符串——不行！timeit 执行的是一个全新函数体，只认识 setup 创建的名字。这是新手误用 timeit 的高频雷区。

21.16 测试排序速度 (Timing sort speed)

英文原文

To demo setup code in API-call mode, the interaction that follows times a sort-based option in Chapter 18's minimum-value example. To avoid page flipping, he

re's the function it times, in the module mins.py (Example 18-2): `def min4(args): return sorted(args)[0]` And here are the results—proving indirectly that sequences sort much faster when they are ordered than they do with randomly ordered contents (to see for yourself, run this in Chapter 18's code folder):

中文翻译

为了在 API 调用模式中演示 setup 代码，下面的交互给第 18 章最小值示例中基于排序的方案计时。为了避免来回翻页，这里给出被计时的函数，它位于模块 mins.py (Example 18-2) 中：`def min4(args): return sorted(args)[0]` 下面是结果——它间接证明：序列在有序时排序比随机顺序内容时快得多（想亲自验证，可以在第 18 章的代码文件夹里运行）：

英文原文 (REPL 运行结果)

```
>>> from timeit import repeat                                     # Standard-lib...
>>> min(repeat(number=1000, repeat=5,                             # Sort an ordered...
        setup='from mins import min4\n'                          # Adjacent string...
        'vals=list(range(1000))',                                # Code within () ...
        stmt='min4(*vals)'))                                     # First import pr...
0.011066518723964691
>>> min(repeat(number=1000, repeat=5,                             # Sort a randomly...
        setup='from mins import min4\n'
        'import random\n'
        'vals=[random.random() for i in range(1000)]',
        stmt='min4(*vals)'))
0.068130933213979
```

中文翻译 (REPL 运行结果)

```
>>> from timeit import repeat                                     #
>>> min(repeat(number=1000, repeat=5,                           # min4
          setup='from mins import min4\n'                       #
          'vals=list(range(1000))',                             # ()
          stmt='min4(*vals)'))                                  # import
0.011066518723964691
>>> min(repeat(number=1000, repeat=5,                           #
          setup='from mins import min4\n'                       #
          'import random\n'                                     #
          'vals=[random.random() for i in range(1000)]',        #
          stmt='min4(*vals)'))                                  #
0.068130933213979
```

深度理解

- 核心概念：这个测试演示了 setup 在 API 模式下的标准写法：多行 setup 用相邻字符串字面量拼接（'from mins import min4\n'vals=...', 括号内可跨行）。结果 0.011 vs 0.068 秒——有序数据比随机数据快 6 倍。
- 底层实现：Python 的排序是 Timsort：它擅长识别"已排序的连续片段"（runs），有序数据近乎 $O(n)$ 扫描即可完成；随机数据则进入完整的归并排序流程。min4 用排序取第一个元素，间接暴露了排序对数据顺序的敏感性。
- 设计原因：作者故意用"排序后取 [0]"而不是 min()，一方面呼应第 18 章的示例，另一方面展示"排序速度随数据顺序变化"这一早前提过的论断。
- 实际问题：真实程序中，如果数据本来就接近有序（如日志时间戳、增量更新的 ID），Timsort 会快得多；反之随机数据的排序成本才是基准值。设计算法时要利用这一特

性。

·初学者误区：以为 `sorted()` 对所有数据耗时恒定；或者以为测试结果可以脱离"数据分布"谈。同样的排序代码，换个数据顺序，耗时差 6 倍。

21.17 自动化 timeit 基准测试 (Automating timeit Benchmarking)

英文原文

Rather than going into more timeit details, let's study a program that deploys it to time both coding alternatives and Python versions. This will let us easily define a set of tests to time in a separate file, and will allow us collect data from multiple Pythons, both individually and grouped (though grouped mode comes with limits today, as you'll see).

中文翻译

与其继续深挖 timeit 的细节，不如研究一个部署它来给"编码方案"和"Python 版本"两者计时的程序。这样我们可以轻松地在独立文件里定义一组待测的测试，并且可以单独或分组地从多个 Python 收集数据（尽管如你接下来会看到的，分组模式今天还有限制）。

深度理解

·核心概念：把"测试定义"（`stmts` 列表）与"测试执行"（`pybench.runner`）分离，是基准测试框架的标准分层：

- 定义与执行解耦，换测试不用改框架，换框架不用改测试。
- 底层实现：pybench 通过 `os.popen` 在 shell 里启动其他 Python 解释器执行 `-m timeit` 命令行，再读取输出——相当于在 Python 里"遥控"其他 Python。
- 设计原因：能对比多个 Python 实现（CPython vs PyPy），正是本书"比较不同 Python"这一视角的落地。
- 实际问题：跨实现自动化的代价是输出格式耦合——一旦某个 Python（如 PyPy）改了 `timeit` 的输出格式，解析代码就崩。工程上这叫"脆弱的外部依赖"。
- 初学者误区：以为"自动化框架写好了就能永久复用"。技术生态在变（PyPy 改 `timeit` 输出就是例证），框架要跟着维护。

21.18 基准测试模块 (Benchmark module)

英文原文

To get started, the module file in Example 21-8, `pybench.py`, is set up to time a sequence of statements coded in scripts that import and use it, with either the Python running its code or all Python versions named in a list. It uses some application-level tools described ahead. Because it mostly applies ideas we've already explored and is amply documented, though, it's listed as mostly self-study material, and an exercise in reading Python code.

中文翻译

先说入门：Example 21-8 的模块文件 `pybench.py` 被设计用来给一串语句计时——这些语句写在导入并使用它的脚本里；它可以只用正在运行其代码的那个 Python，也可以使用列表中列出的所有 Python 版本。它用了一些后面会介绍的应用程序级工具。不过，由于它主要应用我们已经探索过的理念，且文档完备，它被定位为基本上属于自学材料，也是一次阅读 Python 代码的练习。

代码分析

```
r"""
    timeit Python

    1) timeit API Python
    2) os.popen timeit Python -m
       timeit Python
    2 " '

    \t 4

    -
      shell
      shell
    - PyPy 2 timeit

    - setup stmt

    API Python
    timer.py
"""
import sys, os, time, timeit
defnum, defrep = 1000, 5
```

#

```

def show_context():
    """
    f-string
    "... " ' 3.10 PyPy
    """
    print(f"Python {'.'.join(str(x) for x in sys.version_info[:3])}...
          f' on {sys.platform}'
          f" at {time.strftime('%b-%d-%Y, %H:%M:%S')}")

def runner(stmts, pythons=None, tracecmd=False):
    """
    stmts: [(number?, repeat?, stmt-string)]
    pythons: None= python [python ]
    """
    show_context()
    for (number, repeat, stmt) in stmts:
        number = number or defnum          # 0
        repeat = repeat or defrep          # 0
        if not pythons:
            # python stmtAPI
            #
            best = min(timeit.repeat(stmt=stmt, number=number, rep...
            print(f'{best:.4f} {stmt[:70]!r}')
        else:
            # python stmt
            #
            print('-' * 80)
            print(repr(stmt))              #
            for python in pythons:
                stmt = stmt.replace("'", '"')    # " => '
                stmt = stmt.replace('\t', ' ' * 4) # => 4
                lines = stmt.split('\n')         # =>
                args = ' '.join(f'"{line}"' for line in lines) # ...
                oscmd = f'{python} -m timeit -n {number} -r {repea...
                print(oscmd if tracecmd else python)

```

·解释器如何处理：runner 是核心。无 python 时走 API 分支：min(timeit.repeat(...)) 与前面 REPL 用法一致。有 python 时走命令行分支，对每条语句做四步“驯服”：(1) " 全部替换成 ' (避免与 shell 参数引号冲突)；(2) \t 替换成 4 空格（消除制表符在不同 shell 下的不确定性）；(3) 按 \n 拆成一行一个参数；(4) 用 f-string 拼出 python -m timeit -n N -r R "line1" "line2" 命令，os.popen 执行并读取输出。oscmd if tracecmd else python 控制是否打印完整命令。

·设计思想：这是“用 Python 写 shell 自动化”的典型样板：字符串替换、拆行、拼接、子进程调用。docstring 里的 Caveats 就是工程复盘——作者把已知的失败模式写在文档里，是专业软件应有的诚实。

深度理解

·核心概念：pybench 把 timeit 的两种模式统一成一个入口：API 模式测“本机 Python”，命令行模式测“任意列表里的 Python”。两种模式共用同一种测试描述格式 (number, repeat, stmt)。

·底层实现：os.popen(oscmd).read() 会启动一个子 shell 执行命令并捕获标准输出——命令字符串是拼出来的，所以引号、缩进、换行都必须精确。f-string 里 {stmt[:70]}!r 用 !r 转换符输出 repr（带引号的形式），便于复制回命令行验证。

·设计原因：作者把 f-string 的嵌套引号问题（PyPy 3.10

上对' 的处理失败) 也当教材——f"..." 外层双引号内嵌'...' 在标准 CPython 3.12 已允许, 但 PyPy 3.10 实现的是更早的语法规则, 会报错。工具代码也要考虑运行平台的语言版本。

·实际问题: 命令行模式对"嵌入双引号的代码"直接失败 (替换成单引号就改变了语义); PyPy 修改过的 timeit 输出让报告难看——这些"现实阻力"正是文档里 Caveats 存在的意义。

·初学者误区: 在字符串处理代码里随手拼接命令而不考虑 shell 的引号规则; 或者以为 os.popen 的输出可以无条件信任。命令注入、引号转义、输出格式漂移, 都是这类自动化的真实风险。

21.19 基准测试脚本 (Benchmark script)

英文原文

This preceding file is really only half the picture. Testing scripts use this module's function, passing in concrete though variable lists of statements and Pythons to be tested, as appropriate for the usage mode desired. For example, the script in Example 21-9, pybenchtests.py, tests a handful of statements and Pythons by importing and using Example 21-8, and allows command-line arguments to determine part of its operation: `-a` tests all listed Pythons instead of just one, and an added `-t` traces constructed command lines in full so you can see how quotes, multiline statements, and

tabs are handled per timeit's command-line formats shown earlier (see both files' docstrings for more details).

中文翻译

上面这个文件其实只讲了一半。测试脚本使用这个模块的函数，传入具体但可变的语句列表和待测 Python 列表，随所需的用法模式而定。例如 Example 21-9 的脚本 `pybenchtests.py` 通过导入和使用 Example 21-8 来测试若干语句和 Python，并允许命令行参数决定它的一部分行为：`-a` 测试列出的所有 Python 而不仅是当前这一个，额外加的 `-t` 则完整追踪构造出的命令行，让你看到引号、多行语句和制表符如何按照前面展示的 `timeit` 命令行格式被处理（更多细节见两个文件的 docstring）。

代码分析

```
"""
    pybench.py Python
    sys.argv
    <python> pybench_tests.py
        stmts Python

    python3 pybench_tests.py -a
        python stmts
    python3 pybench_tests.py -a -t
        -a
        stmts pythons
    -a Python Python
    REPL "import sys" "sys.executable"
"""
import pybench, sys
```

```

pythons = [
    '/Library/Frameworks/Python.framework/Versions/3.12/bin/python...',
    '/Users/me/Downloads/pypy3.10-v7.3.16-macos_x86_64/bin/pypy3',
]
stmts = [
    #
    (0, 0, '[x ** 2 for x in range(1000)]'), # (,...
    (0, 0, 'res=[]\nfor x in range(1000): res.append(x ** 2)'), #...
    (0, 0, 'list(map(lambda x: x ** 2, range(1000)))'), # \n\t=
    (0, 0, 'list(x ** 2 for x in range(1000))'),
    #
    (0, 0, "s = 'hack' * 2500\nx = [s[i] for i in range(10_000)]"),
    (0, 0, "s = '?'\nfor i in range(10_000): s += '?'"), # PyPy ...
]
tracecmd = '-t' in sys.argv # -t:
pythons = pythons if '-a' in sys.argv else None # -a: ...
pybench.runner(stmts, pythons, tracecmd) # stmts pythons ...

```

·解释器如何处理：sys.argv 是命令行参数列表（脚本名 + 各参数）。'-t' in sys.argv 就是"命令行里有没有 -t"这一布尔判断——用列表成员测试实现简单参数解析。pythons if '-a' in sys.argv else None 把 -a 的有无翻译成 runner 的两种模式：传路径列表 = 命令行模式，传 None = API 模式。stmts 里每组 (0, 0, stmt) 中的 0 表示"用默认值"（defnum=1000、defrep=5），由 runner 里的 number or defnum 兜底替换。

·设计思想：测试定义（stmts/pythons）与执行逻辑完全分离：想加测试，改列表；想换 Python，改路径列表。这是"配置驱动"的极简形式。

深度理解

·核心概念：一个测试脚本 = 数据（stmts/pythons 列表

) + 开关 (sys.argv 里的 -a/-t) + 执行 (runner) 。用 in 判断 argv 就能做出 CLI 选项, 无需 argparse。

·底层实现: sys.argv 里存的是字符串; (0, 0, stmt) 里的 0 是"假值", number or defnum 利用"假值替换"语义把 0 换成默认值——这是 Python 里 or 返回操作数的经典用法。

·设计原因: 把不同代码 (列表推导 vs for 循环 vs map v s 生成器) 和不同字符串操作 (索引遍历 vs 字符串拼接) 放进同一批 stmts, 是为了在一个报告里对比多个维度。

·实际问题: s += '?' 这种循环字符串拼接在 CPython 里因字符串不可变要反复创建新对象, 是最慢的写法之一; PyPy 的 JIT 优化下它反而可能是软肋——这为后面"PyPy 输掉最后一个测试"埋下伏笔。

·初学者误区: 想当然以为 -a 就是"所有 Python 都能测"。跨实现自动化的输出格式耦合问题 (PyPy 改了 timeit 输出) 会让这个模式几乎不可用——工具链的"立场"是设计自动化时必须考虑的。

21.20 对单个 Python 计时 (Timing individual Pythons)

英文原文

The following is the preceding script's output when run to test a specific Python—the one running the script. This mode uses direct timeit API calls, not comma

nd lines, with total time listed in the left column, the statement tested quoted on the right, and a context line at the top courtesy of Python's sys and time modules (see its manuals for more info). These two runs again use CPython 3.12 and PyPy 7.3 (i.e., 3.10), respectively, on the same host:

中文翻译

下面是这个脚本在测试特定 Python——即正在运行脚本的那个 Python——时的输出。这种模式使用直接的 `timeit` API 调用，而不是命令行；左列是总时间，右边带引号的是被测语句，顶部一行是来自 Python `sys` 和 `time` 模块的运行环境信息（更多信息见其手册）。这两次运行依然分别使用 CPython 3.12 和 PyPy 7.3（即 3.10），在同一台主机上：

英文原文（运行结果）

```
$ python3 pybench_tests.py
Python 3.12.2 on darwin at Jun-27-2024, 15:21:02
0.0533 '[x ** 2 for x in range(1000)]'
0.0605 'res=[]\nfor x in range(1000): res.append(x ** 2)'
0.0804 'list(map(lambda x: x ** 2, range(1000)))'
0.0759 'list(x ** 2 for x in range(1000))'
0.4042 "s = 'hack' * 2500\nx = [s[i] for i in range(10_000)]"
0.8061 "s = '?'\nfor i in range(10_000): s += '?'"
$ pypy3 pybench_tests.py
Python 3.10.14 on darwin at Jun-27-2024, 15:22:14
0.0020 '[x ** 2 for x in range(1000)]'
0.0040 'res=[]\nfor x in range(1000): res.append(x ** 2)'
0.0030 'list(map(lambda x: x ** 2, range(1000)))'
0.0077 'list(x ** 2 for x in range(1000))'
```

```
0.0529 "s = 'hack' * 2500\nx = [s[i] for i in range(10_000)]"  
1.2942 "s = '?'\nfor i in range(10_000): s += '?'"
```

中文翻译 (运行结果)

```
$ python3 pybench_tests.py  
Python 3.12.2 on darwin at Jun-27-2024, 15:21:02  
0.0533 '[x ** 2 for x in range(1000)]'  
0.0605 'res=[]\nfor x in range(1000): res.append(x ** 2)'  
0.0804 'list(map(lambda x: x ** 2, range(1000)))'  
0.0759 'list(x ** 2 for x in range(1000))'  
0.4042 "s = 'hack' * 2500\nx = [s[i] for i in range(10_000)]"  
0.8061 "s = '?'\nfor i in range(10_000): s += '?'"  
$ pypy3 pybench_tests.py  
Python 3.10.14 on darwin at Jun-27-2024, 15:22:14  
0.0020 '[x ** 2 for x in range(1000)]'  
0.0040 'res=[]\nfor x in range(1000): res.append(x ** 2)'  
0.0030 'list(map(lambda x: x ** 2, range(1000)))'  
0.0077 'list(x ** 2 for x in range(1000))'  
0.0529 "s = 'hack' * 2500\nx = [s[i] for i in range(10_000)]"  
1.2942 "s = '?'\nfor i in range(10_000): s += '?'"
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值

时机、可变/共享状态等)。

英文原文

Drawing conclusions from this is left as a suggested exercise, but notice that PyPy actually loses to CPython on the very last test run. Again, performance can vary per code, and absolutes in benchmarking are perilous at best.

中文翻译

从中得出结论这件事留作建议练习，但请注意：在最后一个测试里 PyPy 实际上输给了 CPython。再说一次，性能随代码而异，基准测试中的绝对断言最好也就是危险品。

深度理解

- 核心概念：单 Python 模式的输出同时展示了六条语句在一个实现内的相对排名，以及两个实现之间的横向对比。注意最后一列：字符串拼接测试 PyPy 是 1.2942 秒，比 CPython 的 0.8061 秒还慢——PyPy 不是什么都快。
- 底层实现：`s += '?'` 在循环里每次都要创建一个新的字符串对象（字符串不可变），是 $O(n^2)$ 的代价；PyPy 的 JIT 对这类“大对象分配型”循环的优化收益远小于对纯计算的收益，于是败给 CPython。相比之下 `s[i]` 索引遍历测试 PyPy 又快 8 倍 (0.0529 vs 0.4042)。
- 设计原因：作者特意在 `stmts` 里埋了这组字符串测试，就是要打破“PyPy 全面碾压 CPython”的简单叙事——性能结论永远要针对具体代码。

- 实际问题：选择 Python 实现时，要知道自己程序的计算构成：整数/浮点运算密集 → PyPy 赢面大；字符串拼接、C 扩展依赖 → CPython 可能更强。没有免费的午餐。
 - 初学者误区：看到"PyPy 快 20 倍"就断言"PyPy 更好"。同一批测试里 PyPy 就输了最后一项——任何"XX 更快"的说法都要追问一句：测的什么代码？
-

21.21 对多个 Python 计时 (Timing multiple Pythons)

英文原文

As noted, this script can also test multiple Pythons for each statement string, by adding a `-a` to the command line. In this mode the script itself is run by CPython 3.12, which launches shell command lines that start other Pythons to run the `timeit` module on the statement strings. To make this work, this mode must split, format, and quote multiline statements for use in command lines according to both `timeit` expectations and shell requirements. This mode also relies on the `-m` Python command-line flag to locate `timeit` on the module search path and run it as a script, as well as the `os.popen` and `sys.argv` standard-library tools to run a shell command and inspect command-line arguments, respectively. Add a `-t` to trace commands run, and see Python manuals and other resources for more

on these tools; `os.popen` is also mentioned briefly in Chapters 3, 9, and 13. Here is this mode's result:

中文翻译

如前所述，这个脚本还可以通过在命令行加 `-a`，为每条语句字符串测试多个 Python。这种模式下，脚本本身由 CPython 3.12 运行，它启动 shell 命令行来拉起其他 Python，在这些语句字符串上运行 `timeit` 模块。为了让这一切工作，该模式必须按照 `timeit` 的预期和 shell 的要求，把多行语句拆分、格式化并加引号，以便放进命令行。这种模式还依赖 Python 的 `-m` 命令行标志在模块搜索路径上定位 `timeit` 并以脚本方式运行它，以及 `os.popen` 和 `sys.argv` 这两个标准库工具——前者运行 shell 命令，后者检查命令行参数。加 `-t` 可以追踪运行过的命令；更多关于这些工具的信息见 Python 手册和其他资源；`os.popen` 在第 3、9、13 章也简单提到过。下面是这种模式的结果：

英文原文（运行结果）

```
$ python3 pybench_tests.py -a
Python 3.12.2 on darwin at Jun-27-2024, 16:12:39
-----
'[x ** 2 for x in range(1000)]'
/Library/Frameworks/Python.framework/Versions/3.12/bin/python3
1000 loops, best of 5: 52.7 usec per loop
/Users/me/Downloads/pypy3.10-v7.3.16-macos_x86_64/bin/pypy3
WARNING: timeit is a very unreliable tool. use pyperf or somethin...
else for real measurements

pypy3 -m pip install pyperf
pypy3 -m pyperf timeit -n '1000' -r '5' '[x ** 2 for x in range(10...
-----
```

```
1000 loops, average of 5: 2.66 +- 1.55 usec per loop (using standa...
...
more output clipped
...
```

中文翻译 (运行结果)

```
$ python3 pybench_tests.py -a
Python 3.12.2 on darwin at Jun-27-2024, 16:12:39
-----
'[x ** 2 for x in range(1000)]'
/Library/Frameworks/Python.framework/Versions/3.12/bin/python3
1000 loops, best of 5: 52.7 usec per loop
/Users/me/Downloads/pypy3.10-v7.3.16-macos_x86_64/bin/pypy3
WARNING: timeit is a very unreliable tool. use pyperf or somethin...
else for real measurements

pypy3 -m pip install pyperf
pypy3 -m pyperf timeit -n '1000' -r '5' '[x ** 2 for x in range(10...
-----
1000 loops, average of 5: 2.66 +- 1.55 usec per loop (using standa...
...
...
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何

时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Regrettably, this all-Pythons mode is nearly unusable today: the opinionated inserts and mods made to timeit by PyPy render its command-line output very different from the norm, and very difficult to read or list here. Moreover, there's no way to disable these (short of dropping CPython's timeit.py into PyPy's library folder, which seems too much for this demo to ask). Hence, while this script's -a mode still works in this edition, it may be best used to time a set of Pythons that do not subjectively mod the behavior of a widely used standard-library module like timeit.

中文翻译

遗憾的是，这个“所有 Python”模式今天几乎不可用：PyPy 对 timeit 做出的带立场的插入和修改，让它的命令行输出与常规大相径庭，很难在这里阅读或展示。而且，没有办法禁掉这些修改（除非把 CPython 的 timeit.py 丢进 PyPy 的库文件夹——对这个演示来说要求太高了）。因此，虽然这个脚本的 -a 模式在本版中依然能跑，它最好还是用于计时那些不会主观修改 timeit 这类被广泛使用的标准库模块行为的 Python 集合。

深度理解

- 核心概念：-a 模式展示了"跨 Python 自动化基准测试"的现实：框架要同时讨好 timeit 的解析规则和 shell 的引号规则，还要承受被测 Python 对标准库的"个性化改造"。
- 底层实现：此模式的核心链路是 `os.popen(f'{python} -m timeit -n ...')`——PyPy 被拉起后，输出的是它自己的"平均值模式"报告 (`2.66 +- 1.55 usec`)，与 CPython 的 "best of 5" 格式完全不同，两个数在同一个报告里根本没法直接比较。
- 设计原因：这个失败案例本身就是教学点：自动化跨版本对比的最大敌人不是速度，而是输出格式的一致性。不同实现的"统计口径" (`min` vs `mean±std`) 不同，连"谁快"都可能说不清。
- 实际问题：工程上要对比多个 Python 时，更可靠的做法是：统一用 API 模式 (`timeit.repeat`) 在每个 Python 内部运行并自行打印结果，绕开命令行输出的格式差异。
- 初学者误区：以为"自动化"就能解决一切对比问题。事实上工具链的兼容性是自动化的头号风险——把 CPython 的 `timeit.py` 拷贝进 PyPy 库目录这种"硬核操作"都被作者放弃，可见一斑。

21.22 测试集合与字典迭代 (Timing set and dictionary iterations)

英文原文

The good news is that single-Python mode still allows us to easily script a set of benchmarks—even on PyPy. The script in Example 21-10, for instance, uses the driver module in Example 21-8 to see how sets and dictionaries fare.

中文翻译

好消息是，单 Python 模式仍然让我们能轻松地脚本化一组基准测试——即使在 PyPy 上也可以。例如 Example 21-10 的脚本使用 Example 21-8 的驱动模块，看看集合和字典的表现如何。

代码分析

```
import pybench

stmts = [
    #
    (0, 0, '{x ** 2 for x in range(1000)}'),
    (0, 0, 'set(x ** 2 for x in range(1000))'),
    (0, 0, 's=set()\nfor x in range(1000): s.add(x ** 2)'),
    #
    (0, 0, '{x: x ** 2 for x in range(1000)}'),
    (0, 0, 'dict((x, x ** 2) for x in range(1000))'),
    (0, 0, 'd={} \nfor x in range(1000): d[x] = x ** 2'),
]

pybench.runner(stmts, None, False)    # -a
```

·解释器如何处理：六条语句分成三组对照：集合的推导式 / 把生成器传给 set() / for 循环 add；字典的推导式 / 把生成器传给 dict() / for 循环赋值。pybench.runner(stmts, None, False) # -a

ts, None, False) 中 None 表示只用当前 Python, False 表示不追踪命令——全部走 API 模式。

·设计思想：这三组对照精确复刻了前面迭代测试的设计：同一个目标结果，三种构建机制。上一章预告过的"把生成器传给类型名是最慢方案"在这里得到验证。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

As suggested in the preceding chapter, passing a generator to a type name is indeed substantially slower than other construction schemes—as this script's output on both CPython 3.12 and PyPy 7.3 (3.10) finally proves:

中文翻译

正如上一章所建议的，把生成器传给类型名确实比其他构造方案慢得多——这个脚本在 CPython 3.12 和 PyPy 7.3 (3.10) 上的输出终于证明了这一点：

英文原文（运行结果）

```
$ python3 pybench_tests2.py
Python 3.12.2 on darwin at Jun-27-2024, 16:20:19
0.0746 '{x ** 2 for x in range(1000)}'
0.0947 'set(x ** 2 for x in range(1000))'
0.0834 's=set()\nfor x in range(1000): s.add(x ** 2)'
0.0745 '{x: x ** 2 for x in range(1000)}'
0.1174 'dict((x, x ** 2) for x in range(1000))'
0.0754 'd={}\nfor x in range(1000): d[x] = x ** 2'
$ pypy3 pybench_tests2.py
Python 3.10.14 on darwin at Jun-27-2024, 16:22:18
0.0191 '{x ** 2 for x in range(1000)}'
0.0222 'set(x ** 2 for x in range(1000))'
0.0186 's=set()\nfor x in range(1000): s.add(x ** 2)'
0.0188 '{x: x ** 2 for x in range(1000)}'
0.0398 'dict((x, x ** 2) for x in range(1000))'
0.0185 'd={}\nfor x in range(1000): d[x] = x ** 2'
```

中文翻译（运行结果）

```
$ python3 pybench_tests2.py
Python 3.12.2 on darwin at Jun-27-2024, 16:20:19
0.0746 '{x ** 2 for x in range(1000)}'
0.0947 'set(x ** 2 for x in range(1000))'
0.0834 's=set()\nfor x in range(1000): s.add(x ** 2)'
0.0745 '{x: x ** 2 for x in range(1000)}'
0.1174 'dict((x, x ** 2) for x in range(1000))'
0.0754 'd={}\nfor x in range(1000): d[x] = x ** 2'
$ pypy3 pybench_tests2.py
```

```
Python 3.10.14 on darwin at Jun-27-2024, 16:22:18
0.0191 '{x ** 2 for x in range(1000)}'
0.0222 'set(x ** 2 for x in range(1000))'
0.0186 's=set()\nfor x in range(1000): s.add(x ** 2)'
0.0188 '{x: x ** 2 for x in range(1000)}'
0.0398 'dict((x, x ** 2) for x in range(1000))'
0.0185 'd={} \nfor x in range(1000): d[x] = x ** 2'
```

深度理解

·核心概念：集合/字典推导式在 CPython 上几乎与 for 循环持平甚至略快（0.0746 vs 0.0834），而“生成器传给类型名”最慢（0.0947/0.1174）——上一章“生成器慢”的结论在集合、字典上同样成立。

·底层实现：set(gen) 和 dict(gen) 要先把生成器包装进迭代器、再逐项驱动状态机；推导式则有专为集合/字典优化的字节码（BUILDSET/BUILDMAP 加迭代器内联），少了一层生成器的保存/恢复状态开销。

·设计原因：作者再次用“同一套框架、换一批 stmts”就测出新数据——这正是把测试定义外置到脚本（而不是写死在框架里）的红利。

·实际问题：写代码时，{x: f(x) for x in data} 这样的字典推导不仅是表达力问题，在数据量大时也有实打实的速度收益；而 dict((x, f(x)) for ...) 这种“老式写法”要三思。

·初学者误区：把“生成器内存省”与“生成器快”混为一谈——生成器的优势是内存和延迟，不是速度。测速时它几乎总是垫底。

21.23 结论：比较工具 (Conclusion: Comparing tools)

英文原文

If you have time, check out `pybenchtests3.py` in the book example's package (omitted here for space) for another test that verifies that this section's `timeit` tools turn in results similar to the earlier `timer.py` homegrown tools. Its findings generally jive with those in `\Iteration Results\`, though they arrive at them by very different means:

中文翻译

如果你有时间，可以看看书示例包里的 `pybenchtests3.py`（这里因篇幅省略）——那是另一个测试，用来验证这一节的 `timeit` 工具给出的结果与前面 `timer.py` 自制工具相似。它的发现大体上与“迭代测试结果”一节一致，尽管它们是由非常不同的途径得出的：

英文原文 (运行结果)

```
$ python3 pybench_tests3.py
Python 3.12.2 on darwin at Jun-27-2024, 16:26:44
0.2766 "res=[]\nfor x in 'hack' * 2500: res.append(ord(x))"
0.2173 "[ord(x) for x in 'hack' * 2500]"
0.1430 "list(map(ord, 'hack' * 2500))"
0.4172 "list(ord(x) for x in 'hack' * 2500)"
```

中文翻译 (运行结果)

```
$ python3 pybench_tests3.py
Python 3.12.2 on darwin at Jun-27-2024, 16:26:44
0.2766 "res=[]\nfor x in 'hack' * 2500: res.append(ord(x))"
0.2173 "[ord(x) for x in 'hack' * 2500]"
0.1430 "list(map(ord, 'hack' * 2500))"
0.4172 "list(ord(x) for x in 'hack' * 2500)"
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

For more fidelity, study this chapter's code and run more tests on your own. Benchmarks can be great sport, but we'll have to leave further excursions as suggested exercises. Here, it's time to wrap up this chapter and part, so we can move on to learn more about the modules we've been using informally since Chapter 1

6. NOTE — Cross-platform results: As a bonus, folder `benchmark-platform-results` in the book's examples package collects CPython's results for this chapter's timer and pybench tests on multiple platforms—macOS, Windows, Android, and Linux. Due to Python and host differences, not all its results are directly comparable, but they are relatively similar. Surprisingly, an Android foldable phone beats all the PCs, though these tests are CPU bound, and operations like file access may fare differently. Caveat: these results will change; retest with your Pythons and devices for current data.

中文翻译

为了更高的保真度，研究本章的代码，并在你自己的环境里多跑些测试。基准测试可以是绝佳的乐趣，但我们只能把更远的远征留给建议练习。现在，是时候为本章和本部分收尾了，这样我们就能继续学习自第 16 章以来一直在非正式使用的模块。注——跨平台结果：作为彩蛋，书示例包里的 `benchmark-platform-results` 文件夹收集了本章 timer 和 pybench 测试在多个平台上的 CPython 结果——macOS、Windows、Android 和 Linux。由于 Python 和宿主的差异，并非所有结果都直接可比，但它们相对接近。令人惊讶的是，一部安卓折叠屏手机打败了所有 PC——不过这些测试是 CPU 密集型的，像文件访问这样的操作表现可能不同。警告：这些结果会改变；请用你的 Python 和设备重新测试，以获取当前数据。

深度理解

·核心概念：交叉验证——两套完全不同的计时工具（自建 timer 的“手动传函数” vs timeit 的“字符串编译”），在同样的代码上得出同样的相对排名：map > 列表推导 > for 循环 > 生成器。结论不依赖工具，才可信。

·底层实现：两套工具的测量途径不同（函数调用 vs 编译字符串），但“best-of + 固定次数”的统计策略一致，所以相对排名稳定。这正是“同一工具内的比较有意义”的延展：跨工具一致，说明测的是真效应而非工具假象。

·设计原因：作者特意安排了这次交叉验证，因为“自己写的工具测出自己想要的结论”是基准测试的头号自欺风险。两套独立实现相互印证，是科学方法（可复现性）在工程中的体现。

·实际问题：安卓折叠屏跑赢 PC 的趣闻，恰恰说明“CPU 架构、散热、频率管理”对 CPU 密集测试影响之大——也说明跨设备比较必须谨慎。作者的告诫很实在：这些数据会过期，重新测。

·初学者误区：不验证“被测函数输出是否正确”就采信计时结果（本章 runner 的 'All results same.' 检查就是为了堵住这个漏洞）；或者拿不同设备、不同工具的数据混在一起排名。基准测试要“同口径”。

21.24 函数陷阱 (Function Gotchas)

英文原文

Now that we've reached the end of the function story, let's review some common pitfalls. Functions have some jagged edges that you might not expect. They're all relatively obscure, and a few have started to fall away from the language completely in recent releases, but most have been known to trip up new users.

中文翻译

既然我们已经讲完了函数的故事，来回顾一些常见的坑吧。函数有一些你可能想不到的尖角（jagged edges）。它们都相对冷门，其中几个在最近的版本里已经开始从语言中消失，但大多数都出名的绊倒过新用户。

深度理解

- 核心概念：函数部分学完了，但“会写”不等于“不踩坑”。这些陷阱集中在名字解析和对象共享两件事上——它们都由“动态作用域 + 引用语义”这一语言设计直接派生。
- 底层实现：本章陷阱的共同根源：函数编译期决定名字分类（局部/全局/闭包），运行期按引用操作对象。理解这两条，陷阱基本可以推导出来，不用死记。
- 设计原因：作者把陷阱放在函数部分末尾集中讲，是因为它们彼此相关、又都难以用一两句话解释清楚，集中处理比散落各章效果好。
- 实际问题：这些坑在代码评审和调试时高频出现；新版 P

python 的报错信息（如 `UnboundLocalError` 的改进文案）已在缓解，但语义没变。

·初学者误区：遇到 `UnboundLocalError` 就以为 Python 有 bug；或者以为“变量在哪赋值就在哪生效”——实际上分类在编译期就全局（对函数而言）定了。

21.25 局部名字是静态检测的 (Local Names Are Detected Staticly)

英文原文

As you've learned, Python classifies names assigned in a function as locals by default; they live in the function's scope and exist only while the function is running. What you may not realize is that Python detects locals statically, when it compiles the def's code, rather than by noticing assignments as they happen at runtime. This leads to one of the most common oddities posted on the Python newsgroup by beginners. Normally, a name that isn't assigned in a function is looked up in the enclosing module:

中文翻译

正如你已经学到的，Python 默认把函数中被赋值的名字归类为局部（locals）；它们存活于函数的作用域中，只在函数运行期间存在。你可能没意识到的是：Python 是静态地检测局部名字的——在编译 `def` 的代码时——而不是在运

运行时观察赋值何时发生。这导致了初学者贴在 Python 新闻组上的最常见的怪现象之一。通常，函数里没有被赋值的名字会在外层模块中查找：

英文原文 (REPL 演示)

```
>>> X = 99
>>> def selector():
    print(X)                                # X used but not assigned
>>> selector()                             # X found in global scope
99
```

中文翻译 (REPL 演示)

```
>>> X = 99
>>> def selector():
    print(X)                                # X
>>> selector()                             # X
99
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Here, the X in the function resolves to the X in the module. But watch what happens if you add an assignment to X after the reference:

中文翻译

这里，函数中的 X 解析到模块中的 X。但注意：如果在引用之后给 X 加一个赋值，会发生什么：

英文原文 (REPL 演示)

```
>>> def selector():
    print(X)                # Does not yet exist!
    X = 88                  # X classified as a local name (...
>>> selector()              # Can also happen for "import X"...
UnboundLocalError: cannot access local variable 'X' where it is no...
a value
```

中文翻译 (REPL 演示)

```
>>> def selector():
    print(X)                #
    X = 88                  # X
>>> selector()              # "import X" "def X"...
UnboundLocalError: cannot access local variable 'X' where it is no...
a value
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理

解代码实际如何被解释器处理。

- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

You get the name-usage error shown here, but the reason is subtle. Python reads and compiles this code when it's typed interactively or imported from a module. While compiling, Python sees the assignment to `X` and decides that `X` will be a local name everywhere in the function. But when the function is actually run, because the assignment hasn't yet happened when the `print` executes, Python says you're using an undefined name. According to its name rules, it should say this; the local `X` is used before being assigned. In fact, any assignment in a function body makes a name local. Imports, `=`, nested defs, nested classes, and so on are all susceptible to this behavior. The problem occurs because assigned names are treated as locals everywhere in a function, not just after the statements where they're assigned. Really, the previous example is ambiguous: was the intention to print the global `X` and create a local `X`, or is this a real programming error

? Because Python treats X as a local everywhere, it's seen as an error; if you mean to print the global X, you need to declare it in a global statement:

中文翻译

你得到的就是这里展示的名字使用错误，但原因很微妙。Python 在交互式输入或从模块导入时读取并编译这段代码。编译期间，Python 看到对 X 的赋值，于是判定 X 在函数的所有位置都是局部名字。但当函数真正运行时，因为 print 执行时赋值尚未发生，Python 就说你在使用未定义的名字。按照它的名字规则，它理应这么说；局部 X 在使用之前未被赋值。事实上，函数体内任何赋值都会把名字变成局部。import、=、嵌套 def、嵌套类等，全都受此行为影响。问题出在：被赋值的名字在函数中所有位置都被当作局部，而不只是在赋值语句之后。说真的，前面的例子是有歧义的：意图是打印全局 X 再创建局部 X，还是一个真正的编程错误？因为 Python 在所有位置都把 X 当作局部，所以它被看作错误；如果你想打印全局 X，就需要在 global 语句中声明它：

英文原文 (REPL 演示)

```
>>> def selector():
    global X                    # Force X to be global (everywhe...
    print(X)
    X = 88
>>> selector()
99
```

中文翻译 (REPL 演示)

```
>>> def selector():  
    global X                # X  
    print(X)  
    X = 88  
>>> selector()  
99
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Remember, though, that this means the assignment also changes the global X, not a local X. Within a function, you can't use both local and global versions of the same simple name. If you really meant to print the global and then set a local of the same name, you'd need to import the enclosing module and use modul

e attribute notation to get to the global version:

中文翻译

不过请记住，这意味着赋值也会修改全局的 X，而不是某个局部的 X。在一个函数内部，你无法同时使用同一个简单名字的局部版本和全局版本。如果你真的想先打印全局的，再设置同名的局部，就需要导入外层模块并用模块属性记号访问全局版本：

英文原文 (REPL 演示)

```
>>> X = 99
>>> def selector():
    import __main__          # Import enclosing module
    print(__main__.X)        # Qualify to get to global versi...
    X = 88                   # Unqualified X classified as lo...
    print(X)                 # Prints local version of name
>>> selector()
99
88
```

中文翻译 (REPL 演示)

```
>>> X = 99
>>> def selector():
    import __main__          #
    print(__main__.X)        #
    X = 88                   # X
    print(X)                 #
>>> selector()
99
88
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Qualification (the `.X` part) fetches a value from a namespace object. The interactive namespace is a module called `main`, so `main.X` reaches the global version of `X`. If that isn't clear, review Chapter 17. In recent versions, Python has improved on this story somewhat by issuing for this case the more specific `"unbound local"` error message shown in the example listing (it used to simply raise a generic `name error`); this gotcha is still present in general, though.

中文翻译

限定 (qualification, 即 `.X` 部分) 从命名空间对象取一个值。交互式命名空间是一个叫 `main` 的模块，所以 `main.X`

能访问 X 的全局版本。如果不清楚，请复习第 17 章。在最近的版本中，Python 对这个故事有所改善：针对这种情形抛出了示例清单中那个更具体的 `"unbound local"` 错误消息（以前只是抛一个通用的名字错误）；不过这个陷阱总体上依然存在。

深度理解

- 核心概念：局部性判定发生在编译期，不是运行期。函数体里任何位置出现赋值（含 `import`、嵌套 `def`、增强赋值），该名字在整个函数内就被打上“局部”标签，即使引用发生在赋值之前。

- 底层实现：编译 `def` 时，编译器扫描函数体收集被赋值的名字，为每个局部名分配固定的栈槽（slot），字节码用下标访问而不是字典查找——这是局部变量比全局变量快的原因。`print(X)` 编译成 `LOADFAST X (局部)`，而函数运行时该槽还未填充，就报 `UnboundLocalError`；`global X` 则让访问改走 `LOADGLOBAL`。

- 设计原因：“静态局部判定”换来的是性能（槽访问 $O(1)$ ）和确定性；代价是这种反直觉的错误。Python 选择了性能优先，用更清晰的错误消息弥补。

- 实际问题：想“先用全局，再建局部”的需求，Python 明确不支持同名共存——必须用模块属性（`main.X`）显式绕过。这是语言设计划出的边界：简单名字在一个作用域内只能有一种身份。

- 初学者误区：以为错误是“运行时赋值顺序”造成的，把赋值挪到 `print` 之前就完事（其实只要函数里有赋值，`pri`

nt 永远读局部) ; 或者在函数里写了 `X += 1` 这种增强赋值 (它同时是读+写) , 忘记它也会把 `X` 变成局部。

21.26 默认参数与可变对象 (Defaults and Mutable Objects)

英文原文

As noted briefly in Chapters 17 and 18, mutable values for default arguments can retain state between calls, though this is often unexpected. In general, default argument values are evaluated and saved once when a `def` statement is run, not each time the resulting function is later called. Internally, Python saves one object per default argument, attached to the function itself. That's usually what you want—because defaults are evaluated at `def` time, they let you save values from the enclosing scope, if needed (functions defined within loops by factories may even depend on this behavior—see ahead). But because a default retains an object between calls, you have to be careful about changing mutable defaults. For instance, the following function uses an empty list as a default value, and then changes it in place each time the function is called:

中文翻译

正如第 17、18 章简要提到的, 可变 (mutable) 的默认参数值可以在调用之间保留状态——虽然这常常出乎意料。

一般来说，默认参数值在 `def` 语句运行时被求值并保存一次，而不是在之后每次调用生成的函数时重新求值。在内部，Python 为每个默认参数保存一个对象，挂在函数对象本身上。这通常正是你想要的——因为默认值在 `def` 时求值，如果需要，它们让你能把外层作用域的值保存下来（工厂函数在循环里定义的函数甚至可能依赖这个行为——见后文）。但正因为默认值在调用之间保留着一个对象，修改可变默认值就必须小心。例如，下面的函数用空列表作默认值，然后每次调用时都就地修改它：

英文原文 (REPL 演示)

```
>>> def saver(x=[]):           # Saves away a list object
    x.append(1)                 # Changes same object each time!
    print(x)

>>> saver([2])                 # Default not used
[2, 1]

>>> saver()                    # Default used
[1]

>>> saver()                    # Grows on each call!
[1, 1]

>>> saver()
[1, 1, 1]
```

中文翻译 (REPL 演示)

```
>>> def saver(x=[]):           #
    x.append(1)                 #
    print(x)

>>> saver([2])                 #
[2, 1]

>>> saver()                    #
[1]

>>> saver()                    #
[1, 1]
```

```
>>> saver()  
[1, 1, 1]
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Some see this behavior as a feature—because mutable default arguments retain their state between function calls, they can serve some of the same roles as `"static"` local function variables in the C language. In a sense, they work much like global variables, but their names are local to the functions and so will not clash with names elsewhere in a program. To other observers, though, this seems like a gotcha—especially the first time they run into it. There are better ways to retain state between calls in Python (e.g., using the nested scope closures and function attributes we met in t

his part, and the classes we will study in Part VI). Moreover, mutable defaults can be overridden by real values, and are tricky to remember (and to understand at all). They depend upon the timing of default object construction. In the prior example, there is just one list object for the default value—the one created when the def is executed. You don't get a new list every time the function is called, so the list grows with each new append; it is not reset to empty each time.

中文翻译

有些人把这个行为当特性——因为可变默认参数在函数调用之间保留状态，它们可以扮演 C 语言里“静态”局部函数变量（static）的一些角色。从某种意义上说，它们非常像全局变量，但名字对函数来说是局部的，不会与程序中其他地方的名字冲突。但在另一些人看来，这尤其像陷阱——尤其是第一次撞上它的时候。在 Python 里有更好的方式在调用之间保留状态（例如本部分见过的嵌套作用域闭包、函数属性，以及我们将在第六部分学习的类）。而且，可变默认值可以被真实参数覆盖，又很难记住（甚至很难理解）。它们依赖于默认对象构造的时机。在前面的例子里，默认值只有一个列表对象——就是 def 执行时创建的那个。每次调用函数并不会得到新列表，所以列表会随着每次 append 增长；它不会每次被重置为空。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

If that's not the behavior you want, simply make a copy of the default at the start of the function body, or move the default value expression into the function body. As long as the value resides in code that's actually executed each time the function is called, you'll get a new object each time through:

中文翻译

如果这不是你想要的行为，只需在函数体开头复制一份默认值，或者把默认值表达式挪进函数体。只要这个值位于每次调用函数时实际执行的代码里，每次就会得到一个新对象：

英文原文 (REPL 演示)


```
>>> def saver(x=None):           # No argument passed?
    if x is None:
        x = []                   # Run code to make a new list ea...
        x.append(1)              # Changes new list object
        print(x)
>>> saver([2])
[2, 1]
>>> saver()                      # Doesn't grow here
[1]
>>> saver()
[1]
```

中文翻译 (REPL 演示)

```
>>> def saver(x=None):           #
    if x is None:
        x = []                   #
        x.append(1)              #
        print(x)
>>> saver([2])
[2, 1]
>>> saver()                      #
[1]
>>> saver()
[1]
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

By the way, the if statement in this example could almost be replaced by the assignment `x = x or []`, which takes advantage of the fact that Python's `or` returns one of its operand objects: if no argument was passed, `x` would default to `None`, so the `or` would return the new empty list on the right. However, this isn't exactly the same. If an empty list were passed in, the `or` expression would cause the function to extend and return a newly created list, rather than extending and returning the passed-in list like the if version. (The expression becomes `[] or []`, which evaluates to the new empty list on the right; see Chapter 12's "Truth Values Revisited" if you don't recall why.) Real program requirements may call for either behavior.

中文翻译

顺便一提，这个例子中的 if 语句几乎可以用赋值 `x = x or []` 替代——它利用的是 Python 的 `or` 返回其中一个操作数对象这一事实：如果没有传参数，`x` 默认为 `None`，于是 `or` 返回右边那个新的空列表。然而，这并不完全相同。如果传入的是一个空列表，`or` 表达式会让函数扩展并返回一个新创建的列表，而不是像 if 版本那样扩展并返回传入的列表

。（表达式变成 [] or [], 求值为右边新的空列表；想不起为什么，请看第 12 章的“真值回顾”。）真实的程序需求可能要求其中任何一种行为。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Today, another way to achieve the value retention effect of mutable defaults in a possibly less confusing way is to use the function attributes we discussed in Chapter 19:

中文翻译

如今，还有一个也许更不绕的方式来实现可变默认值的“值保留”效果：使用我们在第 19 章讨论过的函数属性（function attributes）：

英文原文 (REPL 演示)

```
>>> def saver():
    saver.x.append(1)
    print(saver.x)
>>> saver.x = []
>>> saver()
[1]
>>> saver()
[1, 1]
>>> saver()
[1, 1, 1]
```

中文翻译 (REPL 演示)

```
>>> def saver():
    saver.x.append(1)
    print(saver.x)
>>> saver.x = []
>>> saver()
[1]
>>> saver()
[1, 1]
>>> saver()
[1, 1, 1]
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

The function name is global to the function itself, but it need not be declared because it isn't changed directly within the function. This isn't used in exactly the same way, but when coded like this, the attachment of an object to the function is much more explicit (and arguably less magical).

中文翻译

函数名对函数自身而言是全局的，但它无需声明，因为在函数内部并没有直接修改它。这种用法与默认参数不完全相同，但按这种方式编码时，对象与函数的挂接关系要显式得多（可以说也少了一些魔法）。

深度理解

- 核心概念：默认值在 `def` 执行时求值一次，之后一直被函数对象持有——这是“可变默认值累积状态”的根源。`def` 是一句执行语句，不是声明。
- 底层实现：函数对象内部保存着 `defaults` 元组（保存位置参数默认值）。`def saver(x=[])` 执行时创建的那个空列表就被存进 `defaults`；之后每次 `saver()` 都把同一个列表对象绑定给局部名 `x`，`x.append` 改的是那个共享对象。

这就是“默认值陷阱”的全部机理。

·设计原因：为什么默认值在 `def` 时求值？一是性能（不用每次调用都求值）；二是刻意支持“工厂函数把外层变量快照进默认值”的用法（第 17 章闭包陷阱的正解）。可变默认值只是这个设计的副作用。

·实际问题：惯用修复是 `x=None + 函数体内 if x is None: x = []`；更现代的替代是闭包/函数属性/类（第六部分）。`x = x or []` 的简写有细微差异（传空列表时行为不同），工程上优先用显式 `if` 版本。

·初学者误区：以为“每次调用都会新建默认列表”；或者看到 `saver([2])` 输出 `[2, 1]` 就困惑——那是传入的真实参数，与默认对象无关。本质是对象共享问题：默认值对象只有一个，所有没传参的调用共享它。

21.27 没有 return 的函数 (Functions Without returns)

英文原文

In Python functions, `return` (and `yield`) statements are optional. When a function doesn't return a value explicitly, the function exits when control falls off the end of the function body. Technically, all functions return a value; if you don't provide a `return` statement, your function returns the `None` object automatically:

中文翻译

在 Python 函数中，`return`（以及 `yield`）语句是可选的。当函数没有显式返回值时，控制流掉出函数体末尾函数就退出。严格来说，所有函数都返回一个值；如果你不写 `return` 语句，函数会自动返回 `None` 对象：

英文原文 (REPL 演示)

```
>>> def proc(x):  
    print(x)                                # No return is a None return  
>>> x = proc('testing 123...')  
testing 123...  
>>> print(x)  
None
```

中文翻译 (REPL 演示)

```
>>> def proc(x):  
    print(x)                                # return None  
>>> x = proc('testing 123...')  
testing 123...  
>>> print(x)  
None
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Functions such as this without a return are Python's equivalent of what are called "procedures" in some languages. They're usually invoked as statements, and the None results are ignored, as they do their business without computing a useful result. This is worth remembering, because Python won't tell you if you try to use the result of a function that doesn't return one. As we noted in Chapter 11, for instance, assigning the result of a list append method won't raise an error, but you'll get back None, not the modified list:

中文翻译

这种没有 return 的函数，是 Python 对某些语言中所谓“过程”（procedures）的等价物。它们通常以语句形式被调用，None 结果被忽略——它们干自己的活，而不计算有用结果。这一点值得记住，因为如果你试图使用一个不返回值的函数的结果，Python 不会提醒你。例如，正如我们在第 11 章指出的：把列表 append 方法的结果赋值给变量不会报错，但你会拿回 None，而不是修改后的列表：

英文原文 (REPL 演示)

```
>>> list = [1, 2, 3]
>>> list = list.append(4)           # append is a "procedure"
>>> print(list)                     # append changes list in place
None
```

中文翻译 (REPL 演示)

```
>>> list = [1, 2, 3]
>>> list = list.append(4)           # append ""
>>> print(list)                     # append
None
```

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

"Common Coding Gotchas" discusses this more broadly. In general, any functions that do their business as a side effect are usually designed to be run as stat

ements, not expressions.

中文翻译

\“常见编码陷阱\”一节更广泛地讨论了这个问题。一般来说，任何靠副作用完成工作的函数，通常都被设计成以语句方式运行，而不是以表达式方式。

深度理解

·核心概念：Python 函数一律返回一个值，没写 return 就隐式返回 None——\“函数\”与\“过程\”（只有副作用、无返回值）的界限在 Python 里是模糊的，靠约定而不是语法区分。

·底层实现：函数体末尾没有 return 时，字节码会附加一个 RETURNCONST None；所以 print(x) 里 x 拿到的是那个唯一的 None 单例对象。is None 判断因此永远可靠。

·设计原因：允许省略 return 是为了简洁（很多函数只是做事的，不需要返回值）；代价是\“误用返回值\”不会报错，只能拿到 None——这是 Python 之\“宽容\”的典型例子。

·实际问题：list = list.append(4) 这种代码把列表变量覆盖成 None，是新手经典事故；链式调用 (list.sort().append(...)) 也会当场失败，因为 sort/append 都返回 None。习惯上：就地修改的方法不要赋值。

·初学者误区：把\“函数没 return\”当错误；或者写 x = mlist.sort() 期望 x 是排好序的列表。正确姿势：要么先 s

ort 再用原变量，要么用返回新列表的 sorted()。

21.28 其他函数陷阱 (Miscellaneous Function Gotchas)

英文原文

Here are two additional function-related gotchas—mostly reviews, but common enough to reiterate. Enclosing scopes and loop variables: We met this gotcha in Chapter 17's discussion of enclosing function scopes, but as a reminder: when coding factory functions (a.k.a. closures), be careful about relying on enclosing function scope lookup for variables that are changed by enclosing loops. When a generated function is later called, all such references will remember the value of the last loop iteration in the enclosing function's scope. In this case, you must use defaults to save loop variable values instead of relying on automatic lookup in enclosing scopes. See "Loops Require Defaults, Not Scopes" in Chapter 17 for more details on this topic. Hiding built-ins by assignment: Also in Chapter 17, we saw how it's possible to reassign built-in names in a closer local or global scope; the reassignment effectively hides ("shadows") and replaces that built-in's name for the remainder of the scope where the assignment occurs. This means you won't be able to use the original built-in value for the name. As long a

s you don't need the built-in value of the name you're assigning, this isn't an issue—many names are built in, and they may be freely reused. However, if you reassign a built-in name your code relies on, you may have problems. So don't do that, unless you really mean to. The good news is that the built-ins you commonly use will soon become second nature, and Python's error trapping will alert you early in testing if your built-in name is not what you think it is.

中文翻译

这里还有两个与函数相关的陷阱——大多是复习，但常见到值得重申。外层作用域与循环变量：我们在第 17 章讲外层函数作用域时遇到过这个陷阱，但提醒一下：编写工厂函数（factory functions，也叫闭包 closures）时，小心不要依赖外层函数作用域的查找来读取被外层循环改变的变量。当生成的函数稍后被调用时，所有这类引用记住的都是外层函数作用域中最后一次循环迭代的值。在这种情况下，你必须用默认值保存循环变量的值，而不是依赖外层作用域的自动查找。更多细节见第 17 章的“循环需要默认值，而不是作用域”。用赋值隐藏内置名：同样在第 17 章，我们看到了如何在更近的局部或全局作用域里重新赋值内置名；这种重新赋值会有效地隐藏（“遮蔽”， shadows）并替换该内置名在赋值所在作用域剩余部分中的名字。这意味着你无法再用该名字的内置值。只要你不需要所赋名字的内置值，这就不是问题——很多名字都是内置的，可以自由重用。然而，如果你重新赋值了代码依赖的内置名，就可能出问题。所以别那么做，除非你真的想。好消息是：你常用的内置

名很快就会成为第二天性，而且如果你的内置名不是你想的那样，Python 的错误捕获会在测试早期就提醒你。

深度理解

·核心概念：两个陷阱都来自第 17 章的“名字解析”规则。闭包陷阱：闭包保存的是单元（cell）引用而非值——循环变量是同一个单元格，最终定格在最后一次迭代的值。遮蔽陷阱：局部赋值会让该名字在整个作用域内不再指向内置对象。

·底层实现：闭包陷阱的机理：外层循环变量 `i` 在编译时被判定为被闭包引用的“自由变量”，函数对象保存对同一个 cell 的引用；循环结束后 cell 里的值就是最后的迭代值，所有闭包读到相同值。修复用默认参数：`lambda i=i: ...` 让每次迭代把当前值快照进默认值。遮蔽陷阱的机理：赋值后字节码对该名字的解析从内置（LOADGLOBAL 链的第三级）改到局部/全局。

·设计原因：闭包按引用（而不是按值）捕获，是“动态词法作用域”的常见实现选择（Lua、JavaScript 同此）；默认参数快照是 Python 给的官方绕行方案。遮蔽则纯粹是“名字只是字典里的键”这一模型的自然推论。

·实际问题：回调列表（按钮绑定循环变量）、装饰器工厂、GUI 事件处理器里，闭包陷阱天天见；`map(lambda x: x, ...)` 里意外覆盖了内置 `map`、`list` 之类的遮蔽事故则在代码评审中高频出现。

·初学者误区：以为闭包会“记住”创建时刻的变量值（实际记住的是变量本身）；以为给 `list` 变量赋过值后还能在

后续代码里用内置 list 函数——遮蔽是“覆盖式”的，一旦赋值，该作用域内内置名就没了。

21.29 本章小结 (Chapter Summary)

英文原文

This chapter rounded out our look at functions and built-in iteration tools with a larger case study that measured the performance of iteration alternatives and other tools we've met along the way, as well as one alternative Python implementation. We used both custom timer code as well as Python's `timeit` with a helper to time code in a variety of modes. We also reviewed common function-related mistakes to help you avoid pitfalls. This concludes the functions part of this book. The next part expands on what we already know about modules—files of tools that form the topmost organizational unit in Python programs, and the structure in which functions reside. After that, we will explore classes, which are largely packages of functions with special first arguments. As you'll see, classes can implement objects that tap into the iteration protocol, just like the generators and iterables we used here. In fact, everything we have learned in this part of the book will apply when functions take the guise of class methods. Before moving on to modules, though, be sure to work through this chapter's quiz, as well as th

e exercises for this part of the book, to practice what you've learned about functions here.

中文翻译

本章用一个更大的案例研究为“函数与内置迭代工具”画上句号：测量了迭代替方案和沿途遇到的其他工具的性能，还测了一种替代 Python 实现。我们既用了自定义计时代码，也用了 Python 的 `timeit` 加辅助工具，以多种模式给代码计时。我们还复习了常见的函数相关错误，帮你避开陷阱。这结束了本书的函数部分。下一部分将扩展我们已经知道的模块（modules）——作为 Python 程序最顶层组织单元的工具文件，也是函数栖身的结构。在那之后，我们将探索类（classes）——大体上就是带特殊第一个参数的函数包。你会看到，类可以实现接入迭代协议的对象，就像我们这里用的生成器和可迭代对象一样。事实上，本书这一部分学到的所有东西，在函数以类方法的面目出现时都会适用。不过在进入模块之前，一定要做完本章的测验和本部分的练习，练习你在这里学到的关于函数的知识。

深度理解

·核心概念：本章三件事串起函数部分：测量工具（自制 `timer` + `timeit`）、测量实践（迭代五种写法、多实现对比）、避坑清单（函数陷阱）。而整章的方法论只有一句：用数据说话。

·底层实现：展望后续：模块就是“名字空间文件”，类方法是“第一个参数绑定成实例的函数”——函数部分学的作用域、参数匹配、闭包，到了模块和类里原样复用。这

就是本书\“概念可迁移\”的编排逻辑。

·设计原因：作者把基准测试放在函数部分末尾，是因为只有掌握了函数、迭代器、生成器，才有资格测量它们；工具与原理齐备后，进入模块部分正好可以\“带着测量工具学模块\”。

·实际问题：学会本章的计时方法，你就能用 `python -m timeit` 或自建 runner 验证自己程序中的\“优化直觉\”——这是工程实践里被低估却极有用的技能。

·初学者误区：把\“测验与答案\”跳过，直接看下一章。本章的 quiz 只有两问，却是对\“结论的前提条件\”（版本、机器、代码类型）的总结性拷问——值得先自答再看答案。

21.30 测验 (Test Your Knowledge: Quiz)

英文原文 (Quiz)

1. What conclusions can you draw from this chapter about the relative speed of Python iteration tools?
2. What conclusions can you draw from this chapter about the relative speed of the Pythons timed?

中文翻译 (测验)

1. 关于 Python 迭代工具的相对速度，你能从本章得出什么结论？
2. 关于被计时的各种 Python 的相对速度，你能从本章得出什么结论？

英文原文 (Answers)

1. In CPython today, list comprehensions are often the quickest of the bunch; map beats list comprehensions in Python when all tools must call built-in functions; for loops tend to be slower than comprehensions; and generator functions and expressions are slower than all other options. Under PyPy, some of these findings differ; map often turns in a different relative performance, for example, and list comprehensions seem always quickest, perhaps due to function-level optimizations. At least that's the case today on the Python versions tested, on the test machine used, and for the type of code timed—these results may vary if any of these three variables differ. Use the homegrown timer or standard library `timeit` to test your use cases for more relevant results. Also keep in mind that iteration is just one component of a program's time: more code gives a more complete picture.

2. In general, PyPy 7.3 (implementing Python 3.10) is substantially faster than CPython 3.12. In some cases timed, PyPy was 5X–20X faster than CPython, though in isolated cases (e.g., some string operations), PyPy was slower than CPython, and PyPy's use of a JIT can impact benchmark results. At least that's the case today on the Python versions tested, on the test machine used, and for the type of code timed—these results may vary if any of these three variables differ. Use the homegrown timer

or standard library `timeit` to test your use cases for more relevant results. This is especially true when timing Python implementations, which may be arbitrarily optimized in each new release—in fact, CPython may soon adopt a JIT like PyPy, which could invalidate results here. We also didn't test any of the many other Python implementations; see Chapter 2 for other options to time on your own.

中文翻译 (答案)

1. 在今天 (today) 的 CPython 上, 列表推导往往是这群方案里最快的; 当所有工具都必须调用内置函数时, `map` 会胜过列表推导; `for` 循环往往比推导式慢; 生成器函数和生成器表达式比所有其他选项都慢。在 PyPy 下, 这些发现有些不同: 例如 `map` 的相对表现往往不同, 而列表推导似乎总是最快——也许归功于函数级优化。至少在被测试的 Python 版本、所用的测试机器、被测代码类型这三者的组合下, 今天的情况就是如此——这三个变量任一个不同, 结果都可能变化。请用自制的 `timer` 或标准库 `timeit` 测你自己的用例, 以获得更相关的结果。还要记住: 迭代只是程序时间的一部分; 测更多代码才能得到更完整的图景。

2. 总的来说, PyPy 7.3 (实现 Python 3.10) 比 CPython 3.12 快得多。在被测的某些情形中, PyPy 比 CPython 快 5 倍到 20 倍; 不过在个别情形 (例如某些字符串操作) 中, PyPy 比 CPython 慢, 而且 PyPy 使用 JIT 会影响基准测试结果。至少在被测试的 Python 版本、所用的测试机器、被测代码类型这三者的组合下, 今天的情况就是如此——这三个变量任一个不同, 结果都可能变化。请用自制的 `timer`

或标准库 `timeit` 测你自己的用例，以获得更相关的结果。给 Python 实现计时尤其如此——每个新版本都可能被任意优化；事实上，CPython 可能很快就会采用像 PyPy 那样的 JIT，这可能会让这里的结果失效。我们也没有测试众多其他 Python 实现中的任何一个；想自己计时其他选项，请看第 2 章。

深度理解

·核心概念：两道测验题的答案共享同一套结论格式：先给“今天的、这台机器上的、这类代码的”观察，再补“三个变量一变，结论就变”的限定，最后给出行动建议（自己测）。这就是科学结论的写法。

·底层实现：答案里的细节都与前文数据一一对应：CPython 上 `map` 在“必须调内置函数”时夺冠（C 层循环）、PyPy 上列表推导总夺冠（JIT 函数级优化）、字符串拼接是 PyPy 的软肋。

·设计原因：作者用“答案即方法论”的方式收尾——学完本章，你带走的不该是几条速度排名，而是“如何正确地得出排名”。

·实际问题：这两问的答案几乎是“基准测试报告的标准模板”：写性能报告、做技术选型文档时，照这个结构写（结论 + 限定条件 + 复现方式）就是专业水准。

·初学者误区：只记住“列表推导最快”这类碎片结论，忽略“today”“测试机器”“代码类型”三个限定词——考试答满限定条件才是本章想要的答案。

21.31 第四部分练习 (Test Your Knowledge: Part IV Exercises)

英文原文

In these exercises, you're going to start coding more sophisticated programs. Be sure to check the solutions in \"Part IV, Functions and Generators\" in Appendix B, and be sure to start writing your code in module files. You won't want to retype these exercises in a REPL if you make a mistake.

1. The basics: At the Python interactive prompt, write a function named `echo` that prints its single argument to the screen and call it interactively, passing a variety of object types: string, integer, list, dictionary. Then, try calling it without passing any argument. What happens? What happens when you pass two arguments?
2. Arguments: Write a function called `adder` in a Python module file named `adder1.py`. The function should accept two arguments and return the sum (or concatenation) of the two. Then, add code at the bottom of the file to call the `adder` function with a variety of object types (two strings, two lists, two floating points), and run this file as a script from the system command line. Do you have to print the call statement results to see results on your screen?
3. Arbitrary arguments: Copy the file you wrote in the last exercise to `adder2.py`, generalize its `adder` function to compute the sum of an arbitrary number

r of arguments, and change the calls to pass more or fewer than two arguments. What type is the returned sum? (Hints: a slice such as `S[:0]` returns an empty sequence of the same type as `S`, and the type built-in function can test types; but see the manually coded `m` in examples in Chapter 18 for a simpler approach.) What happens if you pass in arguments of different types? What about passing in dictionaries?

4. Keywords: In an `adder3.py`, change the `adder` function from exercise 2 to accept and sum/concatenate three arguments: `def adder(red, green, blue)`. Now, provide default values for each argument, and experiment with calling the function interactively or code tests in the file. Try passing one, two, three, and four arguments. Then, try passing keyword arguments. Does the call `adder(blue=1, red=2)` work? Why? Finally, copy and generalize the new `adder` to accept and sum/concatenate an arbitrary number of keyword arguments in an `adder4.py`. This is similar to what you did in exercise 3, but you'll need to iterate over a dictionary, not a tuple. (Hint: the `dict.keys` method returns an iterable you can step through with a `for` or `while`, but be sure to wrap it in a list call to index it; `dict.values` may help here too.)

5. Dictionary tools: Write a function called `copyDict(dict)` that copies its dictionary argument. It should return a new dictionary containing all the items in its argument. Use the dictionary `keys` method to iterate (or step over a dictionary's keys without calling `keys`). C

copying sequences is easy (`X[:]` makes a top-level copy); does this work for dictionaries, too? As explained in this exercise's solution, because dictionaries come with similar tools, this and the next exercise are just coding exercises but still serve as representative functions. 6. Dictionary tools: Write a function called `addDict(dict1, dict2)` that computes the union (i.e., merge) of two dictionaries. It should return a new dictionary containing all the items in both its arguments (which are assumed to be dictionaries). If the same key appears in both arguments, feel free to pick a value from either. Test your function by writing it in a file and running the file as a script. What happens if you pass lists instead of dictionaries? How could you generalize your function to handle this case, too? (Hint: see the type built-in function used earlier.) Does the order of the arguments passed in matter? Dictionary merge is also a built-in today (actually, several), but you're trying to stretch yourself a bit by coding it manually. 7. More argument-matching examples: First, define the following six functions (either interactively or in a module file that can be imported):

```
def f1(a, b): print(a, b) # Normal args
def f2(a, b): print(a, b) # Positional collectors
def f3(a, b): print(a, b) # Keyword collectors
def f4(a, b, c): print(a, b, c) # Mixed modes
def f5(a, b=2, c=3): print(a, b, c) # Defaults
def f6(a, b=2, c): print(a, b, c) # Defaults and positional collectors
```

Now, test the following calls interactive

ly, and try to explain each result; in some cases, you'll probably need to fall back on the matching rules covered in Chapter 18. Do you think mixing matching modes is a good idea in general? Can you think of cases where it would be useful?

```
>>> f1(1, 2) >>> f1(b=2, a=1) >>> f2(1, 2, 3) >>> f3(1, x=2, y=3) >>> f4(1, 2, 3, dict(x=2, y=3)) >>> f5(1) >>> f5(1, 4) >>> f5(1, c=4) >>> f6(1) >>> f6(1, [3, 4])
```

8. Primes revisited: Recall the following code snippet from Chapter 13, which simplistically determines whether a positive integer is prime:

```
x = num // 2 # For some num > 1, start at half while x > 1: if num % x == 0: # Remainder 0? Factor found print(num, 'has factor', x) break # Exit now and skip else x -= 1
```

else: # Normal exit, when x reaches 1 print(num, 'is prime')

Package this code as a reusable function in a module file (num should be a passed-in argument), and add some calls to the function at the bottom of your file to test. While you're at it, experiment with replacing the first line's // operator with / to see how true division breaks this code (refer back to Chapter 5 if you need a reminder). What can you do about negatives, and the values 0 and 1? How about speeding this up? Your outputs should look something like this:

```
13 is prime 13.0 is prime 15 has factor 5 15.0 has factor 5.0
```

9. Iterations and comprehensions: Write code to build a new list containing the square roots of all the numbers in this list: [2, 4, 9, 16, 25]. Code this as a for loop first, then as a map call, then as a list comprehension, and finally as a generator expression. Use the `sqrt` function in the built-in `math` module to do the calculation (i.e., import `math` and say `math.sqrt(X)`). Of the four, which approach do you like best?

10. Timing tools: In Chapter 5, we saw three ways to compute square roots: `math.sqrt(X)`, `X**.5`, and `pow(X,.5)`. If your programs run a lot of these, their relative performance might become important. To see which is quickest, use the `timer2.py` module (Example 21-7) we wrote in this chapter to time each of these three tools. Use its `bestoftotal` function to test. Which of the three square root tools seems to run fastest on your device and Python in general? Finally, how might you use `timer2.py` to interactively time the speed of dictionary comprehensions versus for loops? What about comprehensions with if clauses and nested for loops?

11. Recursive functions: Write a simple recursion function named `countdown` that prints numbers as it counts down to zero. For example, a call `countdown(5)` will print: 5 4 3 2 1 stop. There's no obvious reason to code this with an explicit stack or queue, but what about a non-function approach? Would a generator make sense here?

12. Computing factorials: Finally, a computer-science classic (but demonstrative nonetheless). We em

played the notion of factorials in Chapter 20's coverage of permutations: $N!$, computed as $N \times (N-1) \times (N-2) \times \dots \times 1$. For instance, $6!$ is $6 \times 5 \times 4 \times 3 \times 2 \times 1$, or 720. Code and time four functions that, for a call `fact(N)`, each return $N!$. Code these four functions (1) as a recursive count down per Chapter 19; (2) using the functional `reduce` call per Chapter 19; (3) with a simple iterative counter loop per Chapter 13; and (4) using the `math.factorial` library tool per Chapter 20. Use this chapter's `timeit` to time each of your functions. What conclusions can you draw from your results?

中文翻译

在这些练习中，你将开始编写更复杂的程序。务必对照附录 B 中“第四部分，函数与生成器”的解答，并务必开始把代码写进模块文件。一旦写错，你不会想在 REPL 里重敲这些练习的。

1. 基础：在 Python 交互提示符下，写一个名为 `echo` 的函数，把它的单个参数打印到屏幕，然后交互式调用它，传入各种对象类型：字符串、整数、列表、字典。然后，试着不传任何参数调用它。会发生什么？传两个参数呢？
2. 参数：在名为 `adder1.py` 的 Python 模块文件里写一个叫 `adder` 的函数。函数应接受两个参数并返回两者的和（或拼接）。然后，在文件底部加代码，用各种对象类型（两个字符串、两个列表、两个浮点数）调用 `adder` 函数，并从系统命令行以脚本方式运行该文件。要看到屏幕上的结果，你必须 `print` 调用语句的结果吗？
3. 任意参数：把上一个练习写的文件复制成 `adder2.py`，把它的 `adder` 函数推广为计算任意数量参数的和，并修改调用以传入多于或少于两

个参数。返回的和是什么类型？（提示：像 `S[:0]` 这样的切片会返回与 `S` 同类型的空序列，`type` 内置函数可以测类型；但更简单的做法见第 18 章手工编码的 `min` 示例。）如果你传入不同类型的参数会发生什么？传字典呢？

4. 关键字：在 `adder3.py` 中，把练习 2 的 `adder` 函数改成接受并求和/拼接三个参数：`def adder(red, green, blue)`。现在，给每个参数提供默认值，并尝试交互式调用或在文件里写测试代码。试试传一个、两个、三个、四个参数。然后，试试传关键字参数。调用 `adder(blue=1, red=2)` 能工作吗？为什么？最后，在 `adder4.py` 里复制并推广这个新 `adder`，让它接受并求和/拼接任意数量的关键字参数。这和练习 3 类似，但你需要遍历字典而不是元组。（提示：`dict.keys` 方法返回一个可迭代对象，你可以用 `for` 或 `while` 遍历它，但要索引它时务必用 `list` 调用包住；`dict.values` 这里也可能有用。）

5. 字典工具：写一个叫 `copyDict(dict)` 的函数，复制它的字典参数。它应返回一个包含其参数中所有项的新字典。用字典的 `keys` 方法迭代（或者不调用 `keys` 直接遍历字典的键）。复制序列很容易（`X[:]` 做顶层复制）；这对字典也有效吗？正如本练习解答所解释的，因为字典自带类似工具，这个和下一个练习只是编码练习，但仍是具有代表性的函数。

6. 字典工具：写一个叫 `addDict(dict1, dict2)` 的函数，计算两个字典的并集（即合并）。它应返回一个包含两个参数（假定都是字典）所有项的新字典。如果同一个键在两个参数中都出现，从任意一个取值都可以。把函数写进文件并以脚本方式运行来测试它。如果传列表而不是字典会发生什么？你怎样才能推广你的函数也处理这种情况？（提示：见前面用过的 `type` 内置函数。）传入参数的顺序重要吗？字典合并如今也是内置功能（实际上有好几个），

但你是想通过手工编码来锻炼自己。7. 更多参数匹配示例

：首先，定义下面六个函数（交互式或写进可导入的模块文件都行）：

```
def f1(a, b): print(a, b) # 普通参数
def f2(a, b): print(a, b) # 位置收集
def f3(a, b): print(a, b) # 关键字收集
def f4(a, b, c): print(a, b, c) # 混合模式
def f5(a, b=2, c=3): print(a, b, c) # 默认值
def f6(a, b=2, c): print(a, b, c) # 默认值 + 位置收集
```

然后，交互式测试下面的调用，并试着解释每个结果；有些情况下，你很可能需要回退到第 18 章讲的匹配规则。你认为总体而言混合使用匹配模式是个好主意吗？你能想出它有用的场景吗？

```
>>> f1(1, 2)
>>> f1(b=2, a=1) >>> f2(1, 2, 3) >>> f3(1, x=2, y=3)
>>> f4(1, 2, 3, dict(x=2, y=3)) >>> f5(1) >>> f5(1, 4)
>>> f5(1, c=4) >>> f6(1) >>> f6(1, [3, 4])
```

8. 质数重访：回忆第 13 章的下面这段代码片段，它简单粗暴地判断一个正整数是否为质数：

```
x = num // 2 # 对某个 num > 1, 从一半开始
while x > 1:
    if num % x == 0: # 余数为 0? 找到因子
        print(num, 'has factor', x)
        break # 立即退出并跳过
    else: x -= 1
else: # 正常退出, 即 x 减到 1
    print(num, 'is prime')
```

把这代码打包成模块文件里的可复用函数（num 应该是传入参数），并在文件底部加一些对函数的调用来测试。顺手做个实验：把第一行的 // 运算符换成 /，看看真除法如何弄坏这段代码（需要提醒请看第 5 章）。负数、0 和 1 怎么处理？怎么加速？你的输出大致应该是这样的：

```
13 is prime 13.0 is prime 15 has factor 5 15.0 has factor 5.0
```

9. 迭代与推导式：写代码构建一个新列表，包含此列表中所有数字的平方根：[2, 4, 9, 16, 25]。先写成 for 循环，再写成 map 调用，然后是列表推导，最后是生成器表达式。用内置 math 模块的 sqrt 函数做计算（即 i

import math 后写 `math.sqrt(X)`)。四种方式你最喜欢哪一种？10. 计时工具：在第 5 章，我们见过计算平方根的三种方式：`math.sqrt(X)`、`X .5` 和 `pow(X, .5)`。如果你的程序大量运行这些操作，它们的相对性能就可能变得重要。要看哪个最快，用本章写的 `timer2.py` 模块 (Example 21-7) 给这三种工具分别计时。用它的 `bestoftotal` 函数测试。在你的设备和 Python 上，三种平方根工具哪个总体最快？最后，你如何用 `timer2.py` 交互式地测量字典推导式对比 `for` 循环的速度？带 `if` 子句和嵌套 `for` 循环的推导式呢？11. 递归函数：写一个简单的递归函数 `countdown`，倒数到零时打印数字。例如，调用 `countdown(5)` 会打印：5 4 3 2 1 stop。没有明显的理由用显式栈或队列编码它，但不用函数的方法呢？这里用生成器合理吗？12. 计算阶乘：最后，一个计算机科学经典（虽然只是演示）。我们在第 20 章讲排列时用过阶乘的概念： $N!$ ，按 $N \setminus (N-1) \setminus (N-2) \setminus \dots 1$ 计算。例如 $6!$ 是 $6 \setminus 5 \setminus 4 \setminus 3 \setminus 2 \setminus 1$ ，即 720。编写并计时四个函数，对调用 `fact(N)` 各返回 $N!$ 。四个函数分别按：(1) 第 19 章的递归倒数方式；(2) 第 19 章的函数式 `reduce` 调用；(3) 第 13 章的简单迭代计数循环；(4) 第 20 章的 `math.factorial` 库工具。用本章的 `timeit` 给每个函数计时。你能从结果得出什么结论？

深度理解

·核心概念：12 道练习是函数部分的总验收，主题分布：参数匹配 (2、3、4、7)、作用域 (1、8 含除法陷阱)、迭代与推导 (9、10)、递归与高阶函数 (11、12)、字典工具 (5、6)。其中第 10 题直接让你用本章的 `timer2` 实践计时，第 12 题让你用 `timeit` 对比四种阶乘实现

——学完就能用。

·底层实现：第 3 题的切片 `S[:0]` 技巧利用了空切片与原序列同类型 `S` 的语义；第 8 题的 `//` 换成 `/` 会让 `x` 变成浮点数，`num % x` 的余数语义改变，破坏 `break` 条件——这是运算符语义的实操演练；第 12 题四种阶乘实现（递归/归约/迭代/库函数）恰好构成一个迷你基准测试。

·设计原因：练习刻意覆盖函数部分的全部知识点，并让最后几题直接复用本章刚学的计时工具——教材的 `timeit` 以致用 `timeit` 闭环。

·实际问题：第 10 题是本章计时方法在真实问题（平方根三方案、字典推导 vs `for`）上的直接应用；做完它，你就拥有了用数据回答性能疑问的完整流程。

·初学者误区：直接翻附录 B 抄答案。这些练习的价值在出错—排查—对照的过程；尤其第 7 题的参数匹配，不亲手跑一遍，匹配规则永远记不牢。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

·性能回归防线：把本章的计时方法变成自动化测试（CI 里跑 `python -m timeit` 或 `pyperf`），防止改一次代码性能掉一半无感发生。

·技术选型依据：选迭代写法、选 Python 实现（CPython /PyPy）、选库之前，用本章的\"结论 + 限定条件\"模板写微型基准，代替拍脑袋。

·剖析工具链：微基准之外，全程序性能要用 profile/cProfile（找出真正的热点函数），两者互补——微基准回答\"哪种写法快\"，剖析回答\"该优化哪里\"。

·第三方工具：pyperf（Python 标准基准工具）、pytest-benchmark（把基准写进 pytest）、asv（airspeed velocity，给开源库做跨版本性能追踪）都是生产级选择。

与其他语言/工具的区别

方面	Python	Java	C/C++
标准计时工具	timeit（微基准）、cProfile（剖析）	JMH（基准框架，需要注解和单独构建）	clock()/RDTSC/chrono，自己造轮子
统计策略	多轮取最小值（过滤系统抖动）	JMH 输出吞吐量/均值±误差	取决于实现
JIT 影响	仅 PyPy 等实现；CPython 正在路上	HotSpot 全时 JIT，需 warm-up 处理	无 JIT（除非编译优化）
微基准难度	低（内置模块）	高（JMH 配置繁琐）	高（编译器优化可能删掉被测代码）

核心差别一句话：Python 把\"测速\"做进了标准库，但测出的\"速度\"永远是\"这台机器+这个版本+这段代码\"的组合属性——任何语言都逃不脱这个限定，Python 只是让你测起来最不费力。

Python 发展历史背景

·time.clock 的消亡：Python 3.3 之前测速常用 time.clock

ck() (平台相关、语义混乱) , 3.3 引入 `time.perfcounter/processtime` 后, 3.8 正式移除 `clock`——本章 REPL 例子因此在新版本上“全部崩掉”, 是版本演进的鲜活教材。

- `timeit` 的进化: `timeit` 从 Python 2.3 起就内置, 历经 CLI 选项、`setup` 支持、`globals` 参数等增强; 其“构造 `def` 字符串再编译”的策略长期稳定, 是标准库长寿设计的范例。

- 性能格局的剧变: 2012 年 CPython 3.3 上 `for` 循环最差、生成器居中; 2024 年 CPython 3.12 上排名反转, PyPy 5–20 倍碾压; CPython 3.11 引入自适应特化解释器 (提速约 25%), 未来 JIT 可能再度改写排名——基准数据的“保质期”之短, 本章历史对比就是证明。

- `pypperf` 的崛起: PyPy 团队在 `timeit` 命令行里公开“劝退” (推荐 `pypperf`) , 反映业界对“严谨测速 (方差、统计口径)”的追求正在成为主流。

高级开发者需要掌握的相关知识

- 测速时的“干扰源”清单: CPU 频率缩放/睿频、后台进程、垃圾回收 (可对比 `gc.disable()` 前后的差别)、缓存热效应 (重复执行时数据已在 L1 缓存) 。

- 统计素养: 为什么取 `min` 而非 `mean` (`mean` 对离群值敏感)、什么时候看标准差 (PyPy 的平均值±标准差输出)、如何设计“足够大”的 `number` 让计时分辨率不成为误差来源。

- timeit 的 `globals=globals()` 参数、Timer 类 API、`autorange()` 自动校准次数等进阶用法。
- 装饰器计时器（第 39 章预告）：在“函数已被业务使用”的场景下用 `@timed` 给每次调用注入计时逻辑，与本章“手动传函数”模式的取舍。
- 理解 JIT/解释器差异：为何 PyPy 对纯循环友好而字符串拼接反输，为何 CPython 3.12 的 `map` 对内置函数是“快车道”——从实现细节推导性能，而非死记排名。

学习建议 (Learning Advice)

- 重要程度：★★★★☆（4/5 星）。对“写出正确 Python”不是必需，但对“写出高效的 Python”和“理解 Python 生态”非常重要；作为函数部分的收尾章，它还承载了陷阱复习的功能，不可跳过。
- 应该掌握到什么程度：
 - 必须：会用 `python -m timeit` 命令行和 `timeit.repeat` API；理解 `min(repeat(...))` 为何取最小；知道 `perfcounter` 与 `processtime` 的区别；记住函数三大陷阱（静态局部判定、可变默认值、无 `return` 返回 `None`）并能解释。
 - 应该：能自己写出 `timer.py` 级别的计时模块（`star` 参数、元组返回、`bestoftotal` 嵌套）；理解“被测代码必须被强制求值/构造成本要移出”的测量纪律；能读懂 `pybench` 这种自动化框架。
 - 加分：看懂“结论必须有三个限定条件（版本/机器/代码

类型) \的方法论，并在自己的性能报告中实践；会用 profile/cProfile 做全程序剖析。

·后续应该学习的内容：

·第五部分（下一章起）：模块与包——本章的模块文件（timer.py、pybench.py）正是\“模块作为组织单元\”的提前演示。

·第六部分：类——默认值陷阱的\“正解\”之一是类；类方法就是函数，第 21 章全部知识原样适用。

·第 39 章：装饰器计时器，把本章的计时逻辑改造成 @timed 语法。

·实践项目：给第 12 题（四种阶乘）补上 timeit 计时并写出带限定条件的结论报告；用 pyperf 重测本章的迭代排名。

·纵深：学习 CPython 字节码（dis 模块）解释 map/列表推导/生成器在字节码层面的差异——这是\“为什么 map 快、生成器慢\”的最深层答案。